

Angular Learning Series

Topics 48 – 53

Testing Services & HTTP · Angular CDK · Angular Universal & SSR
NgRx Core Concepts · NgRx Effects · NgRx Selectors In Depth

Generated: May 17, 2026

TOPIC 48 — Testing Services & HTTP

Angular Testing | Intermediate | Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — You already know how to test components (Topic 47) and how HttpClient works (Topic 36), so the core challenge here is understanding a new testing-specific infrastructure layer (the HTTP mock backend) rather than a brand-new concept.

Angular Relevance: Critical — Every Angular app that talks to an API has services with HTTP calls; untested service logic is the most common source of production regressions because bugs hide in transform logic, error paths, and header handling that no component test ever reaches.

Section 1 — Explanation

The Post Office Intercept Analogy

Imagine your service is a person who writes letters (HTTP requests) and sends them to the post office. In production, the post office actually delivers the letters and brings back real replies. In testing, you don't want real letters going anywhere — so you set up a fake post office clerk who sits at the counter, intercepts every letter before it leaves the building, lets you read what was written, and hands back a scripted reply you prepared in advance.

HttpClientTestingModule is that fake post office. HttpTestingController is the clerk — you ask it "did anyone send a letter to this address?", verify what the letter said, and then flush back a fake reply. Your service never knows the difference.

Spies are a separate concept: they sit on a real or fake object's method and record every call — arguments, return value, call count — without necessarily replacing the method's behaviour. They let you assert that something was called with something, not just what came back.

Technical Explanation

Angular's HttpClientTestingModule replaces the real HttpBackend with HttpClientTestingModuleBackend. When your service calls this.http.get(url), no network request leaves the browser. Instead the request is queued inside HttpTestingController. Your test then:

1. Triggers the service method
2. Calls httpController.expectOne(url) to assert the request was made and get a handle to it
3. Calls req.flush(mockData) to send back a fake response
4. The Observable in your service emits mockData as if the server replied
5. Calls httpController.verify() after each test to assert no unexpected requests were made

Tool	What it does	When to use
HttpClientTestingModule	Replaces real HTTP backend	Every service test that uses HttpClient

Tool	What it does	When to use
HttpTestingController	Inspects & flushes queued requests	Assert URL, method, headers, body
req.flush(data)	Sends a success response	Happy path tests
req.flush(msg, {status:500})	Sends an error response	Error path tests
httpController.verify()	Asserts no unexpected requests remain	Always in afterEach
jasmine.createSpy()	Creates a standalone spy function	When you need to track calls
spyOn(obj, method)	Wraps an existing method with a spy	When mocking service dependencies

Section 2 — Connection to Prior Concepts

Directly builds on:

- **Topic 36 — HttpClient & API Integration:** The service code being tested here is exactly what you wrote in Topic 36.
- **Topic 47 — Testing Angular Components:** The TestBed.configureTestingModule() pattern, fixture.detectChanges(), fakeAsync/tick, and { provide: X, useValue: mock } patterns all carry over directly.
- **Topic 37 — HTTP Interceptors:** Understanding that interceptors run in the real backend chain explains why HttpClientTestingModule bypasses them by default.
- **Topic 18 — Error Handling:** Testing the catchError path in your service requires flushing error responses and asserting on the caught result.

Bridging note: In Topic 47, you mocked entire services with useValue objects so components never made real HTTP calls. Here you go one level deeper — you are testing the service itself, so you can't mock the service away. Instead you mock only the HTTP transport layer while letting the real service logic run. This is the critical distinction: component tests mock services; service tests mock HTTP.

Most directly extends: Topic 47 — Testing Angular Components.

Section 3 — Code Example

Plain TS — Core concept: spy recording calls

```
// Understanding spies in isolation before Angular context
const calculator = {
  add: (a: number, b: number) => a + b
};

// Wrap the real method with a spy – it still calls through by default
const spy = spyOn(calculator, 'add').and.callThrough();
calculator.add(2, 3); // => 5, but spy records the call
```

```

console.log(spy.calls.count()); // 1
console.log(spy.calls.first().args); // [2, 3]
console.log(spy.calls.first().returnValue); // 5
// Replace behaviour entirely
spyOn(calculator, 'add').and.returnValue(99);
calculator.add(2, 3); // => 99 – real method never called

```

The Service Under Test

```

// src/app/services/user.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpResponseError } from '@angular/common/http';
import { Observable, catchError, throwError, map } from 'rxjs';
export interface User { id: number; name: string; email: string; }
export interface UserListResponse { users: User[]; total: number; }
@Injectable({ providedIn: 'root' })
export class UserService {
  readonly #http = inject(HttpClient);
  readonly #baseUrl = 'https://api.example.com';
  getUsers(): Observable<User[]> {
    return this.#http
      .get<UserListResponse>(`${this.#baseUrl}/users`)
      .pipe(
        map(response => response.users), // transform: strip wrapper
        catchError(this.#handleError)
      );
  }
  getUserById(id: number): Observable<User> {
    return this.#http
      .get<User>(`${this.#baseUrl}/users/${id}`)
      .pipe(catchError(this.#handleError));
  }
  createUser(user: Omit<User, 'id'>): Observable<User> {
    return this.#http
      .post<User>(`${this.#baseUrl}/users`, user)
      .pipe(catchError(this.#handleError));
  }
  #handleError(error: HttpResponseError): Observable<never> {
    const message =
      error.status === 0
        ? 'Network error – check your connection'
        : `Server error: ${error.status}`;
    return throwError(() => new Error(message));
  }
}

```

The Complete Test File

```

// src/app/services/user.service.spec.ts
import { TestBed } from '@angular/core/testing';
import {

```

```

HttpClientTestingModule,
HttpTestingController
} from '@angular/common/http/testing';
import { UserService, User, UserListResponse } from '../user.service';
describe('UserService', () => {
  let service: UserService;
  let httpController: HttpTestingController;
  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
    });
    service = TestBed.inject(UserService);
    httpController = TestBed.inject(HttpTestingController);
  });
  afterEach(() => {
    httpController.verify();
  });
  it('should GET /users and unwrap the users array', () => {
    const mockResponse: UserListResponse = {
      users: [
        { id: 1, name: 'Alice', email: 'alice@example.com' },
        { id: 2, name: 'Bob', email: 'bob@example.com' }
      ],
      total: 2
    };
    let result: User[] | undefined;
    service.getUsers().subscribe(users => (result = users));
    const req = httpController.expectOne('https://api.example.com/users');
    expect(req.request.method).toBe('GET');
    req.flush(mockResponse);
    expect(result).toEqual(mockResponse.users);
    expect(result?.length).toBe(2);
  });
  it('should surface a readable error message on 500', () => {
    let caughtError: Error | undefined;
    service.getUsers().subscribe({
      next: () => fail('Expected an error, got success'),
      error: (err: Error) => (caughtError = err)
    });
    const req = httpController.expectOne('https://api.example.com/users');
    req.flush('Internal Server Error', { status: 500, statusText: 'Server Error' });
    expect(caughtError?.message).toBe('Server error: 500');
  });
});

```

Section 4 — Before & After Refactor

WRONG — Using HttpClientModule instead of HttpClientTestingModule

```
// WRONG: Using HttpClientModule – actual network calls!
describe('UserService (broken)', () => {
  let service: UserService;
  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientModule], // Real HTTP backend
    });
    service = TestBed.inject(UserService);
  });
  // No afterEach verify() – extra requests silently pass
  it('should get users', () => {
    service.getUsers().subscribe(users => {
      expect(users.length).toBeGreaterThan(0);
      // Observable is async – assertion may never run!
    });
  });
});
```

CORRECT — HttpClientTestingModule with verify()

```
describe('UserService (correct)', () => {
  let service: UserService;
  let httpController: HttpTestingController;
  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
    });
    service = TestBed.inject(UserService);
    httpController = TestBed.inject(HttpTestingController);
  });
  afterEach(() => httpController.verify());
  it('should get users', () => {
    let result: User[] | undefined;
    service.getUsers().subscribe(users => (result = users));
    const req = httpController.expectOne('https://api.example.com/users');
    req.flush({ users: [{ id: 1, name: 'Alice', email: 'a@test.com' }], total: 1 });
    expect(result?.length).toBe(1);
  });
});
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Forgetting httpController.verify() in afterEach

What it looks like: Tests pass even when the service makes unexpected extra HTTP requests.

Why developers make it: They focus on the happy path and never read about verify().

How to avoid it: Always add `afterEach(() => httpController.verify())` as the very first thing after `beforeEach`.

Angular-specific consequence: A service that fires two GET requests instead of one will pass all tests silently. In production, users get double API calls and potential race conditions.

Mistake 2 — Asserting inside subscribe without ensuring the Observable emitted

What it looks like: Assertions inside subscribe callbacks that never run because `flush()` was never called.

Why developers make it: They come from Promise-based testing where `.then()` assertions work differently.

How to avoid it: Always declare a result variable outside subscribe, flush the request, then assert on the variable after `flush()` — never inside the callback alone.

Angular-specific consequence: Tests that assert nothing are silently green. A completely broken `getUsers()` method passes all its tests with zero failures.

Mistake 3 — Using expectOne with a partial URL or wrong URL

What it looks like: Service calls `/users?page=1` but test uses `expectOne('/users')` — throws "found none".

Why developers make it: They write the base URL without query parameters because that's what they think they're testing.

How to avoid it: Use a matcher function: `httpController.expectOne(r => r.url === '/users' && r.params.get('page') === '1')`.

Angular-specific consequence: The test throws during setup with a misleading error — developers debug the service when only the test URL is wrong.

Section 6 — Do's and Don'ts

DO	DON'T
Import <code>HttpClientTestingModule</code> in every service test	Import <code>HttpClientModule</code> — it makes real network calls
Call <code>httpController.verify()</code> in <code>afterEach</code>	Rely on individual tests to catch unexpected HTTP requests
Declare result variables outside subscribe, assert after <code>req.flush()</code>	Assert only inside the subscribe callback
Test both happy path and error path (flush with <code>{ status: 500 }</code>)	Only test success responses
Assert <code>req.request.method</code> , <code>req.request.body</code> , <code>req.request.headers</code>	Assert only on the response — verify what was sent
Use matcher function <code>expectOne(req => ...)</code> for dynamic query params	Hardcode a URL with query params as a string

Section 7 — Debugging Tips

Bug 1 — "Expected one matching request for criteria ..., found none"

Symptom: Test throws during `httpController.expectOne(url)`, not inside any assertion.

Cause: The service didn't make an HTTP request to that exact URL. URL has a typo, method was never called, or query parameters were appended.

Fix: Use `httpController.match(() => true)` to capture all requests and log their URLs, then fix `expectOne` to match exactly.

Angular note: `HttpParams` encodes values and appends them — `{ page: 1 }` becomes `?page=1` in the full URL.

Bug 2 — Test passes but assertion inside `subscribe` was never reached

Symptom: Test is green, but adding `expect(true).toBe(false)` inside `subscribe` also passes.

Cause: You triggered the service method but never called `req.flush()` to make the `Observable` emit.

Fix: Always flush the request after calling the service method — `flush()` is what makes `subscribe` fire.

Angular note: `HttpClientTestingModule` queues requests synchronously but only emits when you explicitly flush.

Bug 3 — "1 request(s) pending after verify()"

Symptom: `afterEach` throws after a test that otherwise passed.

Cause: Your service made an HTTP request you never handled with `expectOne` + `flush`.

Fix: Add `httpController.match(() => true)` before `verify()` temporarily to see what URL is pending, then decide whether to `expectOne` it or fix the service.

Angular note: This error often reveals a real production bug — your service is making a call you didn't intend.

Section 8 — Real-World Applications

Scenario 1 — Testing an Auth Token Interceptor

```
describe('AuthInterceptor', () => {
  let httpController: HttpTestingController;
  let http: HttpClient;
  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [
        provideHttpClient(withInterceptors([authInterceptor]))
      ]
    });
  });
  http = TestBed.inject(HttpClient);
```



```

httpController = TestBed.inject(HttpTestingController);
localStorage.setItem('token', 'fake-jwt-token');
});
afterEach(() => { httpController.verify(); localStorage.clear(); });
it('should attach Authorization header to every request', () => {
http.get('/api/protected').subscribe();
const req = httpController.expectOne('/api/protected');
expect(req.request.headers.get('Authorization'))
.toBe('Bearer fake-jwt-token');
req.flush({});
});
it('should NOT attach Authorization header to login endpoint', () => {
http.post('/api/auth/login', {}).subscribe();
const req = httpController.expectOne('/api/auth/login');
expect(req.request.headers.has('Authorization')).toBeFalse();
req.flush({ token: 'new-token' });
});
});

```

Scenario 2 — Testing a Caching Service

```

it('should only make one HTTP call even when called twice', () => {
const mockProducts: Product[] = [{ id: 1, name: 'Widget' }];
service.getProducts().subscribe();
const req = httpController.expectOne('/api/products');
req.flush(mockProducts);
// Second call – should return cached data, no HTTP
service.getProducts().subscribe();
// verify() will throw if a second request was made
httpController.verify();
const pendingRequests = httpController.match('/api/products');
expect(pendingRequests.length).toBe(0);
});

```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a `WeatherService` with a single `getCurrentWeather(city: string)` method that calls `GET /api/weather?city={city}`. Write a test that verifies: (a) the correct URL including query parameter is called, (b) the returned data matches the flushed mock, and (c) `verify()` passes cleanly.

Exercise 2 — Intermediate

Create a `NotificationService` with `markAsRead(id)` calling `PATCH /api/notifications/{id}` with body `{ read: true }` and `deleteAll()` calling `DELETE /api/notifications`. Write tests verifying: (a) correct HTTP method, (b) correct PATCH body, (c) a 403 response surfaces a typed error, (d) no unexpected requests are made.

Exercise 3 — Advanced

Build a `SearchService` that debounces by 400ms (`debounceTime`), calls `GET /api/search?q={term}`, cancels in-flight requests on new search (`switchMap`), and transforms raw `{ results, meta }` response to return only results. Write a full test suite using `fakeAsync/tick` for debounce behaviour, `HttpTestingController` for the request and transformation, and flush an error response to verify the error path.

Section 10 — Key Takeaways

- `HttpClientTestingModule` replaces the real HTTP backend — no network calls leave your test environment
- `HttpTestingController` is your inspector — use it to assert what was requested and flush back controlled responses
- `req.flush()` is what makes HTTP Observables emit in tests — without it, the subscription callback never fires
- Spies track method calls — arguments, return values, call count — without necessarily replacing behaviour
- `verify()` is your safety net — it catches requests that were made but never asserted on

Scenario	Tool	Code
Match exact URL	<code>expectOne(string)</code>	<code>httpController.expectOne('/api/users')</code>
Match URL with dynamic params	<code>expectOne(fn)</code>	<code>expectOne(r => r.url.includes('/users/'))</code>
Send success response	<code>req.flush(data)</code>	<code>req.flush({ users: [] })</code>
Send error response	<code>req.flush(msg, opts)</code>	<code>req.flush(", { status: 404, statusText: 'Not Found' })</code>
Assert all requests handled	<code>verify()</code>	<code>afterEach(() => controller.verify())</code>
Spy on existing method	<code>spyOn</code>	<code>spyOn(service, 'method').and.returnValue(of(mock))</code>

Angular-specific rules:

- Always use `HttpClientTestingModule`, never `HttpClientModule`, in service tests
- To test interceptors, register them with `provideHttpClient(withInterceptors([...]))` — `HttpClientTestingModule` alone does not run interceptors
- `req.flush()` triggers synchronous emission — no `fakeAsync` needed for basic HTTP tests
- For debounce or timer-based operators, you do need `fakeAsync + tick(400)`

One-Line Memory Aid: Fake clerk intercepts every letter, you check what was written and hand back a scripted reply — `HttpClientTestingModule` is the fake post office, `HttpTestingController` is the clerk, `req.flush()` is the scripted reply.

Section 11 — Thought-Provoking Question + Answer

Question: If `HttpClientTestingModule` fully replaces the HTTP backend, why do your interceptors not run by default in service tests — and what does this tell you about where interceptors actually live in Angular's HTTP pipeline?

Answer: Interceptors in Angular are registered at the `HttpClient` provider level, not at the `HttpBackend` level. When you use bare `HttpClientTestingModule`, Angular configures a basic `HttpClient` with no interceptors registered. The interceptors you wrote in your app config are part of a different provider configuration in your app module — your test module is a separate injector with no knowledge of your app's interceptors unless you explicitly add them.

This means service unit tests and interceptor integration tests are different things. A service test should test the service's own logic in isolation — you don't want interceptors running because a bug in an interceptor would break your service tests. An interceptor test should use `provideHttpClient(withInterceptors([yourInterceptor]))` to opt into only that interceptor.

Practical rule: Test interceptors separately by explicitly registering them. Keep service tests interceptor-free unless specifically testing the interaction between a service and an interceptor.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts

- Mocking vs Stubbing vs Spying — understanding the three terms precisely helps you choose the right Jasmine tool for each testing scenario

TypeScript Concepts

- Conditional types & type narrowing in tests — writing `expect(result).toBeDefined()` before asserting on `result!.property`

Angular-Specific Concepts

- Topic 49 — Angular CDK — CDK introduces new services with their own testing utilities
- Testing Effects and Signals — Signals used inside services need `TestBed.flushEffects()`
- Component + Service integration tests — the natural next step after unit-testing each in isolation

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 36 — `HttpClient`, Topic 37 — HTTP Interceptors, Topic 47 — Testing Angular Components, Topic 18 — Error Handling, Topic 20 — RxJS Subscription Management.

Topics this unlocks: Topic 49 — Angular CDK (CDK services have their own test harnesses), Topic 50 — Angular Universal & SSR (SSR tests need platform token mocking).

Production readiness assessment: A developer who masters service testing with `HttpClientTestingModule` and spies can confidently test the majority of Angular business logic layers in a real production app — this is the highest-ROI testing skill after component testing.

Section 14 — Common Interview Questions

Question 1 (Mid): "What is HttpClientTestingModule and why do we need it instead of just mocking the entire service?"

Strong answer covers: It replaces only the HTTP transport layer, not the service — so the service's own transformation and error handling actually runs; without it, tests make real network calls; `HttpTestingController` lets you assert on the request (URL, method, headers, body); it makes HTTP Observables emit synchronously via `flush`.

Weak answers miss: The distinction between mocking the service entirely (for component tests) vs mocking only HTTP (for service tests).

Question 2 (Mid-Senior): "Why might a test using HttpClientTestingModule pass even though the service method has a bug?"

Strong answer covers: Assertions inside `subscribe` never run if `flush()` was never called; `verify()` not being called means extra requests are ignored; asserting on the wrong URL in `expectOne`; not testing the error path.

Weak answers miss: The "silent green test" problem — Jasmine passes tests with zero assertions.

Question 3 (Senior): "How would you test a service that uses `debounceTime(400)` inside a `switchMap` to debounce search API calls?"

Strong answer covers: Wrap in `fakeAsync`; call the service method, then `tick(400)` to advance the virtual timer; only after `tick(400)` will `expectOne` find the queued request; demonstrate that calling twice within 400ms only produces one request.

Weak answers miss: The interaction between `fakeAsync/tick` and `debounceTime`.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Asserting only inside subscribe with no external variable

```
// WRONG
it('should return 2 users', () => {
  service.getUsers().subscribe(users => {
    expect(users.length).toBe(2); // silently never runs
  });
  const req = httpController.expectOne('/api/users');
  // Missing req.flush() – Observable never emits
});

// CORRECT
it('should return 2 users', () => {
  let result: User[] | undefined;
  service.getUsers().subscribe(users => (result = users));
  const req = httpController.expectOne('/api/users');
  req.flush({ users: [mockUser1, mockUser2], total: 2 });
  expect(result?.length).toBe(2);
});
```

Anti-Pattern 2 — No afterEach verify, relying on per-test cleanup

Why it seems fine: Each test individually passes or fails. What breaks: When a refactor causes services to make extra calls, verify() in unrelated tests starts throwing — developers blame the wrong test.

Correct pattern: Always use afterEach(() => httpController.verify()) globally.

Anti-Pattern 3 — Using spyOn to mock HttpClient methods directly

```
// WRONG
it('should call http.get', () => {
  const httpSpy = spyOn(TestBed.inject(HttpClient), 'get')
  .and.returnValue(of(mockData));
  service.getUsers().subscribe();
  expect(httpSpy).toHaveBeenCalledWith('/api/users');
});
// Bypasses all pipe() operators – catchError, map, retry never run
```

Correct pattern: Always use HttpClientTestingModule + HttpTestingController. It tests what the service does through its entire pipeline.

Section 16 — Mental Model Summary

Core concept: HttpClientTestingModule swaps the real HTTP transport for a fake one that queues requests for your inspection, while HttpTestingController lets you assert on what was sent and flush back controlled responses to drive your service's Observable pipeline.

Key rules:

1. Always import HttpClientTestingModule, never HttpClientModule, in service tests
2. Always call httpController.verify() in afterEach — catches extra requests
3. Declare result variables outside subscribe; assert after req.flush(), never only inside the callback
4. Test the error path by flushing with { status: 4xx/5xx }
5. Use expectOne(fn) for URLs with dynamic query parameters
6. To test interceptors, register them explicitly — HttpClientTestingModule alone does not run them

THE SINGLE MOST IMPORTANT RULE: req.flush() is what makes the Observable emit — without it, every assertion inside subscribe silently never runs and the test passes as a false green.

TOPIC 49 — Angular CDK (Overlay, DragDrop, VirtualScroll)

Angular CDK | Intermediate | High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The CDK builds on concepts you already own (services, DI, directives, lifecycle hooks), but introduces a new mental model: UI behaviour as injectable infrastructure rather than component-specific code.

Angular Relevance: High — The CDK is Angular's official toolkit for building accessible, themeable UI primitives; understanding it prevents you from reaching for heavy third-party libraries for overlay menus, drag-and-drop boards, and large dataset rendering.

Section 1 — Explanation

The IKEA Flat-Pack Analogy

Imagine Angular Material is a fully assembled sofa — beautiful, ready to use, but you can't change the frame or the fabric without a fight. The Angular CDK is the flat-pack version — it gives you all the structural components (the screws, the frame, the mechanism) but zero visual styling. You assemble what you want, skin it however your design system requires, and the CDK handles all the hard behavioural logic: positioning an overlay correctly as the window scrolls, tracking pointer coordinates during a drag, rendering only the visible rows of a 50,000-item list.

Three Core CDK Modules

- **Overlay** — positions floating panels (tooltips, dropdowns, modals) relative to a trigger element and manages their lifecycle
- **DragDrop** — turns any list or grid into a drag-and-drop interface with transfer between lists
- **VirtualScroll** — renders only the items currently visible in the viewport, making enormous lists feel instant

Module	Key Import	Core Directive/Service	Output/Event
Overlay	OverlayModule	Overlay service	OverlayRef
DragDrop	DragDropModule	cdkDrag, cdkDropList	(cdkDropListDropped)
VirtualScroll	ScrollingModule	CdkVirtualScrollViewpo rt	scrolledIndexChange

Section 2 — Connection to Prior Concepts

- **Topic 27 — Services & Dependency Injection:** The Overlay service is injected exactly like any other Angular service.

- **Topic 41 — Custom Directives:** `cdkDrag` and `cdkDropList` are directives — understanding how directives attach behaviour to host elements explains how the CDK enhances your markup.
- **Topic 32 — ViewChild/ContentChild:** `CdkVirtualScrollViewport` is often queried with `@ViewChild` to programmatically scroll to an index.
- **Topic 45 — Performance Optimisation:** `CdkVirtualScrollViewport` is the CDK's answer to the render-time performance problem.

Bridging note: In Topic 41 you built directives that modify a host element's behaviour directly. The CDK's `cdkDrag` directive does the same thing at industrial scale — it attaches pointer listeners, manages state, and communicates with sibling `cdkDropList` directives through a shared service injected into both.

Most directly extends: Topic 41 — Custom Directives.

Section 3 — Code Example

Virtual Scroll Component

```
// product-list.component.ts
import { Component, signal } from '@angular/core';
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [ScrollingModule],
  template: `
<cdk-virtual-scroll-viewport
  itemSize="60"
  minBufferPx="120"
  maxBufferPx="240"
  class="viewport">
<div
  *cdkVirtualFor="let product of products(); trackBy: trackById"
  class="product-row">
<span>{{ product.name }}</span>
<span>{{ product.price | currency }}</span>
</div>
</cdk-virtual-scroll-viewport>
`,
  styles: [`.viewport { height: 400px; overflow-y: auto; }
.product-row { height: 60px; display: flex; align-items: center; }`]
})
export class ProductListComponent {
  products = signal(
    Array.from({ length: 50_000 }, (_, i) => ({
      id: i, name: `Product ${i}`, price: Math.random() * 100
    })))
};
```

```
trackById = (_, number, item: any) => item.id;
}
```

Drag and Drop — Kanban Board

```
// kanban-board.component.ts
import { Component, signal } from '@angular/core';
import { CdkDragDrop, DragDropModule, moveItemInArray, transferArrayItem } from '@angular/cdk/drag-drop';

@Component({
  selector: 'app-kanban-board',
  standalone: true,
  imports: [DragDropModule],
  template: `
<div cdkDropList id="todo-list"
[cdkDropListData]="todo()"
[cdkDropListConnectedTo]="['done-list']"
(cdkDropListDropped)="onDrop($event)">
<div *ngFor="let task of todo(); trackBy: trackById" cdkDrag>
{{ task.title }}
</div>
</div>
<div cdkDropList id="done-list"
[cdkDropListData]="done()"
[cdkDropListConnectedTo]="['todo-list']"
(cdkDropListDropped)="onDrop($event)">
<div *ngFor="let task of done(); trackBy: trackById" cdkDrag>
{{ task.title }}
</div>
</div>
`
})
export class KanbanBoardComponent {
  todo = signal([
    { id: 1, title: 'Design' },
    { id: 2, title: 'Build' }
  ]);
  done = signal([
    { id: 3, title: 'Plan' }
  ]);
  trackById = (_, number, t: any) => t.id;
  onDrop(event: CdkDragDrop<any[]>): void {
    if (event.previousContainer === event.container) {
      const updated = [...event.container.data];
      moveItemInArray(updated, event.previousIndex, event.currentIndex);
      event.container.id === 'todo-list'
        ? this.todo.set(updated)
        : this.done.set(updated);
    } else {
      const prev = [...event.previousContainer.data];
      const curr = [...event.container.data];
      transferArrayItem(prev, curr, event.previousIndex, event.currentIndex);
      event.previousContainer.id === 'todo-list'
        ? (this.todo.set(prev), this.done.set(curr))
        : (this.done.set(prev), this.todo.set(curr));
    }
  }
}
```



```
}  
}
```

Overlay — Custom Dropdown

```
// dropdown.component.ts  
import { Component, inject, TemplateRef, ViewChild, ViewContainerRef, OnDestroy }  
from '@angular/core';  
import { Overlay, OverlayModule, OverlayRef, ConnectedPosition } from '  
@angular/cdk/overlay';  
import { TemplatePortal } from '@angular/cdk/portal';  
@Component({  
  selector: 'app-dropdown',  
  standalone: true,  
  imports: [OverlayModule],  
  template: `  
    <button #triggerEl (click)="toggleDropdown()">Options</button>  
    <ng-template #dropdownPanel>  
      <div class="dropdown-panel">  
        <button (click)="selectOption('Edit')">Edit</button>  
        <button (click)="selectOption('Delete')">Delete</button>  
      </div>  
    </ng-template>  
  `,  
})  
export class DropdownComponent implements OnDestroy {  
  readonly #overlay = inject(Overlay);  
  readonly #vcr = inject(ViewContainerRef);  
  @ViewChild('triggerEl') triggerEl!: any;  
  @ViewChild('dropdownPanel') dropdownPanel!: TemplateRef<unknown>;  
  #overlayRef: OverlayRef | null = null;  
  toggleDropdown(): void {  
    this.#overlayRef?.hasAttached() ? this.close() : this.open();  
  }  
  open(): void {  
    const positions: ConnectedPosition[] = [  
      { originX: 'start', originY: 'bottom', overlayX: 'start', overlayY: 'top' },  
      { originX: 'start', originY: 'top', overlayX: 'start', overlayY: 'bottom' }  
    ];  
    this.#overlayRef = this.#overlay.create({  
      positionStrategy: this.#overlay.position()  
        .flexibleConnectedTo(this.triggerEl)  
        .withPositions(positions),  
      scrollStrategy: this.#overlay.scrollStrategies.reposition(),  
      hasBackdrop: true,  
    });  
    this.#overlayRef.backdropClick().subscribe(() => this.close());  
    this.#overlayRef.attach(new TemplatePortal(this.dropdownPanel, this.#vcr));  
  }  
  close(): void { this.#overlayRef?.detach(); }
```

```
selectOption(opt: string): void { console.log(opt); this.close(); }
ngOnDestroy(): void { this.#overlayRef?.dispose(); }
}
```

Section 4 — Before & After Refactor

WRONG — Rendering 50,000 items with standard *ngFor

```
// All 50,000 divs created and painted immediately – 3-8 second render
<div class="scroll-container">
  <div *ngFor="let item of items; trackBy: trackById" class="row">
    {{ item.name }}
  </div>
</div>
```

CORRECT — Virtual scroll renders only visible rows

```
// Fixed-height viewport – only ~12 items in DOM at any time
<cdk-virtual-scroll-viewport itemSize="50" class="viewport">
  <div *cdkVirtualFor="let item of items(); trackBy: trackById" class="row">
    {{ item.name }}
  </div>
</cdk-virtual-scroll-viewport>
// CSS: .viewport { height: 600px; overflow-y: auto; }
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Not giving cdk-virtual-scroll-viewport a fixed height

The viewport must have a fixed CSS height. Without it, the CDK calculates 0px visible area and renders nothing — or renders all items, defeating the purpose entirely.

Angular consequence: Virtual scroll silently renders zero items. Developers debug the data source when the problem is one missing CSS rule.

Mistake 2 — Calling moveItemInArray directly on a signal's array

moveItemInArray mutates the array IN PLACE. Signal's internal reference hasn't changed — OnPush components never update. Always spread: `const updated = [...tasks()]; moveItemInArray(updated, ...); tasks.set(updated)`.

Angular consequence: The drag animation completes, the item snaps back to its original position — mutation happened in memory but Change Detection never ran.

Mistake 3 — Forgetting to dispose() the OverlayRef on component destroy

detach() removes the content visually but does not remove the overlay container element from the DOM or unsubscribe event listeners. dispose() must be called in ngOnDestroy.

Angular consequence: Each time the component is created and destroyed, a new detached overlay container accumulates in document.body — event listener leaks and eventual z-index conflicts.

Section 6 — Do's and Don'ts

DO	DON'T
Give <code>cdk-virtual-scroll-viewport</code> a fixed CSS height	Leave viewport without explicit height and wonder why nothing renders
Spread into a new array before <code>moveItemInArray</code> , then <code>signal.set()</code>	Call <code>moveItemInArray</code> directly on <code>signal()</code> — mutations don't trigger reactivity
Call <code>overlayRef.dispose()</code> in <code>ngOnDestroy</code>	Rely on Angular's cleanup to remove overlay DOM nodes automatically
Use <code>scrollStrategies.reposition()</code> for triggered dropdowns	Use <code>scrollStrategies.noop()</code> — panel drifts from trigger on scroll
Use <code>withPositions([primary, fallback])</code> for screen-edge safety	Hardcode a single position — dropdown disappears on small viewports
Import <code>ScrollingModule</code> , <code>DragDropModule</code> , <code>OverlayModule</code> separately	Import an entire CDK mega-import — adds unused code to bundle

Section 7 — Debugging Tips

Bug 1 — Virtual scroll renders nothing (blank viewport)

Cause: The viewport has no height. Fix: Add explicit CSS height. Verify with `scrolledIndexChange` output binding.

Bug 2 — Dragged item snaps back after drop

Checklist: (1) Verify `[cdkDropListData]="yourArray()"` is bound. (2) Verify `(cdkDropListDropped)` is on `cdkDropList` not `cdkDrag`. (3) Verify spreading: `const updated = [...event.container.data]`. (4) Verify `signal.set(updated)` is called.

Bug 3 — Overlay panel appears in wrong position or behind other elements

Wrong position: `flexibleConnectedTo()` received wrong element — check `@ViewChild` resolves to the actual trigger. Behind elements: A parent has `transform/filter` CSS — these create a stacking context and trap the overlay. Remove `transform` from parent or use `cdkOverlayOrigin` directive.

Section 8 — Real-World Applications

Scenario 1 — Infinite Audit Log Table (VirtualScroll + HTTP)

An admin dashboard displaying 200,000+ entries uses `CdkVirtualScrollViewport` with a custom `DataSource` that fetches pages of 100 entries as the user scrolls near the bottom (listening to `scrolledIndexChange`). Only the visible rows are ever in the DOM.

Scenario 2 — Reorderable Dashboard Widget Grid (DragDrop + persisted state)

A SaaS dashboard lets users drag widget tiles into custom order using `cdkDropList` with `cdkDropListOrientation="mixed"`. On drop, the service calls `PATCH /widgets/order` to persist the

new arrangement. Custom `*cdkDragPreview` and `*cdkDragPlaceholder` templates provide visual feedback.

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a standalone `ContactListComponent` rendering 10,000 contacts using `CdkVirtualScrollViewport`. Each row must be exactly 56px tall. Viewport must be 500px tall. Verify smooth scrolling and minimal DOM nodes via browser DevTools.

Exercise 2 — Intermediate

Build a priority queue component with three `cdkDropList` columns: High, Medium, Low. Users can drag tasks between all three and reorder within each. Use Signals for all state. Ensure `moveItemInArray` and `transferArrayItem` are called on spread copies.

Exercise 3 — Advanced

Build a custom `TooltipDirective` using the CDK Overlay service that: shows a tooltip panel on mouseenter, hides on mouseleave, positions above the host with fallback to below, uses `scrollStrategies.close()` so tooltip hides on scroll, and calls `overlayRef.dispose()` on directive destroy. Accept a `[tooltip]` input for text content.

Section 10 — Key Takeaways

- The CDK provides unstyled, accessible UI behaviour primitives — you own HTML/CSS, CDK owns interaction logic
- `CdkVirtualScrollViewport` renders only visible rows — requires fixed-height viewport and `itemSize` in pixels
- `cdkDrag` + `cdkDropList` handle all pointer tracking and placeholder rendering — you handle only data updates in `onDrop`
- Overlay positions floating panels correctly relative to trigger elements outside your component tree
- All three modules are tree-shakable — import only what you use

Feature	Required setup	Key gotcha
VirtualScroll	Fixed viewport height, <code>itemSize</code> prop	No height = zero items rendered
DragDrop	<code>cdkDropList</code> on container, <code>cdkDrag</code> on items, <code>[cdkDropListData]</code> bound	Mutating signal array in place = snap-back on drop
Overlay	Overlay service, <code>OverlayRef</code> , <code>TemplatePortal</code>	No <code>dispose()</code> = DOM node leak on each component destroy

One-Line Memory Aid: CDK gives you the invisible mechanism behind UI magic — VirtualScroll recycles DOM nodes, DragDrop tracks your pointer, Overlay owns position logic — you own the styles and the data.

Section 11 — Thought-Provoking Question + Answer

Question: CdkVirtualScrollViewport requires a fixed itemSize in pixels — but what do you do when your list items have variable heights?

Answer: The default strategy uses uniform size — it assumes every item is exactly itemSize pixels tall so it can calculate any item's scroll offset with simple multiplication ($O(1)$). This breaks for variable heights.

Three production strategies: (1) Force uniform heights — constrain cards to a maximum height with "Show more". Most performant and common. (2) Paginate instead of virtualise — for genuinely variable content like rich text feeds, load-more pagination often delivers better UX. (3) Implement a custom VirtualScrollStrategy with a position map that updates as items are rendered.

Practical rule: Use CdkVirtualScrollViewport when you control item height (data tables, contact lists, log viewers). For user-generated content with variable height, use pagination or load-more patterns instead.

Section 12 — Related Topics to Learn Next

- **Pointer Events API (JS)** — understanding pointerdown/pointermove/pointerup explains how cdkDrag tracks dragging
- **Generic constraints and DataSource<T> (TS)** — the CDK's DataSource class uses generics extensively
- **Topic 50 — Angular Universal & SSR** — CDK modules use document and window internally; SSR requires platform-specific guards
- **Angular Material** — Material components are built on CDK primitives; understanding CDK explains why Material's MatSelect, MatDialog, and MatTable behave as they do
- **CDK A11y module** — FocusTrap, LiveAnnouncer, and FocusMonitor complement Overlay-based components to meet WCAG requirements

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 27 — DI, Topic 32 — ViewChild, Topic 41 — Custom Directives, Topic 45 — Performance Optimisation, Topic 23 — Angular Signals.

Topics this unlocks: Topic 50 — Angular Universal & SSR (CDK uses browser APIs requiring SSR guards). Angular Material deep dives.

Production readiness assessment: A developer who understands all three CDK features can replace the majority of heavy UI library dependencies with zero-dependency, design-system-compatible CDK primitives.

Section 14 — Common Interview Questions

Question 1 (Mid): "When would you use `CdkVirtualScrollViewport` and what are its limitations?"

Strong answer: Use for large lists (500+ items) where DOM node count degrades scroll performance; requires fixed `itemSize`; viewport must have explicit CSS height; variable-height workarounds include experimental `AutoSizeVirtualScrollStrategy` or pagination.

Weak answers miss: The `itemSize` fixed-height requirement and what to do when items vary in height.

Question 2 (Mid): "You implement drag-and-drop and after every drop the item snaps back. Walk me through debugging."

Strong answer: Check `[cdkDropListData]` binding; check `(cdkDropListDropped)` is on `cdkDropList` not `cdkDrag`; check `moveItemInArray` is called on a spread copy; check `signal.set()` is called with new reference; check `event.container.data` is defined.

Weak answers miss: The signal/immutability issue.

Question 3 (Senior): "Explain how the CDK Overlay service positions a dropdown and why you need multiple position entries."

Strong answer: `flexibleConnectedTo(trigger)` anchors panel to trigger's bounding rect; CDK tries each position in order and picks the first that fits within viewport; without fallback, dropdown near bottom clips off-screen; scroll strategies control panel behaviour on scroll.

Weak answers miss: The fallback position array purpose.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using `*ngFor` for large lists when `*cdkVirtualFor` is appropriate

Why it seems fine: Works in development with small dataset. What breaks: With 5,000-20,000 items, first render takes 4-10 seconds, scroll drops to 10fps, mobile browser tabs crash.

Fix: Use `CdkVirtualScrollViewport` + `*cdkVirtualFor` for any list that may exceed ~200 items in production.

Anti-Pattern 2 — Creating a new `OverlayRef` on every open without checking if one already exists

Why it seems fine: Dropdown appears and behaves correctly on first use. What breaks: Each call appends a new overlay container and registers new scroll/resize/backdrop listeners. After 20 open/close cycles, 20 orphaned containers and 20 active listeners degrade performance.

Fix: Guard with `if (this.#overlayRef?.hasAttached())` return; before creating a new ref.

Anti-Pattern 3 — Using `cdkDropListConnectedTo` with string IDs and template method calls for dynamic lists

Why it seems fine: Works for small boards. What breaks: `getConnectedIds()` as a template method call runs on every CD cycle. With 10 columns and frequent drags, fires hundreds of

times per second.

Fix: Use `@ViewChildren` to get `CdkDropList` references and compute connected lists with a `computed()` signal.

Section 16 — Mental Model Summary

Core concept: The Angular CDK provides the behaviour and positioning logic for complex UI interactions — `VirtualScroll`, `DragDrop`, and `Overlay` — while leaving all visual styling entirely to you.

Key rules:

1. `cdk-virtual-scroll-viewport` must have fixed CSS height and matching `itemSize` — no height means no rendering
2. Always spread array before `moveItemInArray` / `transferArrayItem` with Signals — mutations don't trigger reactivity
3. Call `overlayRef.dispose()` in `ngOnDestroy` — `detach()` alone leaves orphaned DOM nodes
4. Provide fallback positions in `withPositions([primary, fallback])` for screen-edge safety
5. Import only the CDK modules you use — `ScrollingModule`, `DragDropModule`, `OverlayModule` separately

THE SINGLE MOST IMPORTANT RULE: `moveItemInArray` and `transferArrayItem` mutate arrays in place — always spread into a new array first, then call `signal.set()`, or your drag-and-drop will silently snap back on every drop.

TOPIC 50 — Angular Universal & SSR

Angular SSR | Advanced | High

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — SSR introduces a fundamentally different execution environment (Node.js, no browser APIs) running the same Angular code, requiring you to reason about where your code runs, not just what it does.

Angular Relevance: High — SSR directly impacts SEO, first contentful paint, and Core Web Vitals scores; in any Angular app where public discoverability or perceived load speed matters, SSR is the production answer — and Angular 17+ ships it as a first-class option via `@angular/ssr`.

Section 1 — Explanation

The Theatre Understudy Analogy

Imagine your Angular app is a play. Normally the full cast performs live on stage for the audience (the browser) — the audience sees nothing until the actors are ready, the set is built, and the lights come up. This is Client-Side Rendering (CSR).

SSR is like hiring an understudy company to pre-perform the first scene on a sound stage (the server) and photograph it. When the real audience arrives, they instantly see a high-quality photo of the first scene (the server-rendered HTML). While they're looking at the photo, the real cast quietly takes their positions. When the cast is ready, the photo is swapped for the live performance seamlessly — this is called hydration.

Concept	CSR	SSR
First paint	Blank until JS loads	Instant HTML from server
SEO	Poor (crawlers see empty shell)	Excellent (full HTML)
Server required	No (static hosting)	Yes (Node.js or edge)
Browser APIs available	Yes (window, document)	No — crash if used without guards
Hydration	N/A	Required — attaches events to server HTML

Section 2 — Connection to Prior Concepts

- **Topic 27 — Services & DI:** `isPlatformBrowser` and `PLATFORM_ID` are injected tokens; `HttpClient` in SSR uses a different backend provider injected at the server platform level.
- **Topic 36 — `HttpClient`:** HTTP calls during SSR run in Node.js, not the browser — understanding the service layer is essential for controlling when fetches happen.

- **Topic 25 — Component Lifecycle Hooks:** `ngOnInit` runs on the server; `ngAfterViewInit` does not reliably produce a DOM on the server.
- **Topic 49 — Angular CDK:** CDK modules use document and window directly — every CDK feature requires `isPlatformBrowser` guard in SSR.

Bridging note: Throughout Topics 1–49 you wrote code with one implicit assumption: there is a browser. SSR removes that assumption entirely. Every technique you've learned still works — but code that touches browser globals must now be wrapped in a platform check, injected through Angular's abstraction layer, or deferred to the client.

Most directly extends: Topic 27 — Services & Dependency Injection.

Section 3 — Code Example

Adding SSR to an existing Angular 17+ app

```
ng add @angular/ssr
# Generates:
# server.ts – Express app serving Angular SSR
# app.config.server.ts – Server-specific providers
# Modifies angular.json to add the server build target
```

app.config.ts — Shared config (runs on both server AND browser)

```
import { provideHttpClient, withFetch } from '@angular/common/http';
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withFetch()), // withFetch() required – Node has no XHR
  ]
};
```

app.config.server.ts — Server-only config

```
import { mergeApplicationConfig } from '@angular/core';
import { provideServerRendering } from '@angular/platform-server';
import { appConfig } from './app.config';
const serverConfig = {
  providers: [provideServerRendering()]
};
export const config = mergeApplicationConfig(appConfig, serverConfig);
```

Guarding browser-only code

```
import { Component, inject, OnInit, PLATFORM_ID } from '@angular/core';
import { isPlatformBrowser, isPlatformServer } from '@angular/common';
@Component({ selector: 'app-product-detail', template: '...' })
export class ProductDetailComponent implements OnInit {
  readonly #platformId = inject(PLATFORM_ID);
  ngOnInit(): void {
    // Safe on both – HttpClient is abstracted
    this.#productService.getProduct(1).subscribe(p => this.product.set(p));
  }
}
```

```

if (isPlatformBrowser(this.#platformId)) {
  // Only runs in browser – safe to access window/document
  const scrollPos = window.scrollToY;
  localStorage.setItem('lastViewed', '1');
}
if (isPlatformServer(this.#platformId)) {
  console.log('Rendering on server');
}
}
}
}

```

afterNextRender() — Modern browser-only hook

```

import { Component, afterNextRender, ElementRef, inject } from '@angular/core';
@Component({ selector: 'app-chart', template: '<canvas #chartCanvas></canvas>' })
export class ChartComponent {
  readonly #el = inject(ElementRef);
  constructor() {
    // afterNextRender NEVER runs on the server – no guard needed
    afterNextRender(() => {
      const canvas = this.#el.nativeElement.querySelector('canvas');
      this.#initChart(canvas); // Safe: real browser DOM API
    });
  }
}

```

TransferState — Preventing duplicate HTTP requests

```

import { Injectable, inject, TransferState, makeStateKey, PLATFORM_ID } from '@angular/core';
import { isPlatformServer } from '@angular/common';
import { of } from 'rxjs';
import { tap } from 'rxjs/operators';
const PRODUCTS_KEY = makeStateKey<Product[]>('products');
@Injectable({ providedIn: 'root' })
export class ProductService {
  readonly #http = inject(HttpClient);
  readonly #ts = inject(TransferState);
  readonly #platformId = inject(PLATFORM_ID);
  getProducts(): Observable<Product[]> {
    if (this.#ts.hasKey(PRODUCTS_KEY)) {
      const cached = this.#ts.get(PRODUCTS_KEY, []);
      this.#ts.remove(PRODUCTS_KEY); // remove so later navigations re-fetch
      return of(cached); // no HTTP call – browser already has data
    }
    return this.#http.get<Product[]>('/api/products').pipe(
      tap(products => {
        if (isPlatformServer(this.#platformId)) {
          // Server embeds this in the HTML as a JSON script tag
          this.#ts.set(PRODUCTS_KEY, products);
        }
      })
    );
  }
}

```

```
}  
);  
}  
}
```

Section 4 — Before & After Refactor

WRONG — Unsafe browser API access

```
// CRASHES with ReferenceError: localStorage is not defined  
@Component({ selector: 'app-theme-toggle', template: '...' })  
export class ThemeToggleComponent implements OnInit {  
  ngOnInit(): void {  
    const saved = localStorage.getItem('theme') ?? 'light'; // CRASH on server  
    document.body.classList.add(saved); // CRASH on server  
  }  
}
```

CORRECT — Platform-guarded browser access

```
@Component({ selector: 'app-theme-toggle', template: '...' })  
export class ThemeToggleComponent implements OnInit {  
  readonly #platformId = inject(PLATFORM_ID);  
  readonly #document = inject(DOCUMENT); // Angular's safe document abstraction  
  ngOnInit(): void {  
    if (isPlatformBrowser(this.#platformId)) {  
      const saved = localStorage.getItem('theme') ?? 'light';  
      this.#document.body.classList.add(saved);  
    }  
    // On server: skip silently – default theme renders  
  }  
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Using localStorage/window without a platform guard

Why developers make it: Works perfectly in development because ng serve runs CSR by default. The SSR server is only used in production builds.

Angular consequence: Server crashes with ReferenceError, returns 500 to every user, negating all SSR benefits.

Mistake 2 — Making duplicate HTTP requests on the client (forgetting TransferState)

Why developers make it: The app visually works correctly — the browser re-fetches and re-renders. Developers don't notice unless they open the Network panel.

Angular consequence: Every SSR page load makes double the API requests — doubles infrastructure cost, users on slow connections see a flash of loading state after hydration.

Mistake 3 — Expecting ngAfterViewInit to safely access the DOM during SSR

Why developers make it: The mental model "AfterViewInit = DOM is ready" is correct in the browser. Developers assume the same guarantee applies on the server.

Angular consequence: Server crashes with `TypeError` — server-side DOM stubs (domino) don't support canvas, media, or other browser-specific APIs.

Section 6 — Do's and Don'ts

DO	DON'T
Use <code>isPlatformBrowser(inject(PLATFORM_ID))</code> to guard browser-only code	Access <code>window</code> , <code>localStorage</code> , or <code>document</code> directly
Use <code>afterNextRender()</code> for DOM-dependent third-party library init	Use <code>ngAfterViewInit</code> for code requiring real browser canvas/media APIs
Use <code>inject(DOCUMENT)</code> instead of raw <code>document</code> global	Use <code>document.querySelector()</code> directly
Add <code>withFetch()</code> to <code>provideHttpClient()</code> — required for Node.js HTTP	Use <code>provideHttpClient()</code> without <code>withFetch()</code> — XHR does not exist in Node
Use <code>TransferState</code> to prevent double HTTP fetching	Ignore transfer state and accept doubled API requests on every SSR page load
Test SSR locally with <code>npm run serve:ssr</code>	Test only with <code>ng serve</code> — CSR dev server hides SSR-specific crashes

Section 7 — Debugging Tips

Bug 1 — `ReferenceError: localStorage is not defined`

Symptom: Server crashes, returns 500. Terminal shows `ReferenceError`.

Fix: Search codebase for `localStorage`, `sessionStorage`, `window.`, `document.` — every hit needs `isPlatformBrowser` guard or `afterNextRender()`. For third-party libraries: lazy-load inside `afterNextRender(() => import('library'))`.

Bug 2 — Hydration mismatch warning: NG0500 or NG0501

Symptom: Browser console shows NG0500: Angular hydration was requested but server-rendered content is missing.

Cause: HTML server rendered doesn't match what Angular expects to hydrate. Common causes: `Math.random()`, `Date.now()`, or browser-conditional rendering.

Fix: Remove non-deterministic values from initial render. Use `ngSkipHydration` on components that intentionally differ.

Bug 3 — Infinite redirect loop on the server for authenticated routes

Cause: Route guard reads `localStorage` for auth token — on server `localStorage` is undefined, guard fails, redirects to `/login` — login guard also fails, infinite loop.

Fix: Return true from guards on the server (let browser re-guard after hydration), or read auth token from HTTP request cookie using injected REQUEST token.

Section 8 — Real-World Applications

Scenario 1 — E-commerce Product Page (SEO-critical SSR)

Product page must be indexable by Google and load instantly. HTTP call runs on server — product data is in HTML before JS loads. TransferState prevents re-fetching in browser. "Add to Cart" uses isPlatformBrowser guard since it accesses localStorage.

Scenario 2 — SSR with Cookie-Based Authentication

```
// In server.ts – passing the Express request to Angular's DI
server.get('**', async (req, res, next) => {
  commonEngine.render({
    bootstrap,
    url: `${req.protocol}://${req.headers.host}${req.originalUrl}`,
    providers: [
      { provide: REQUEST, useValue: req }, // inject req into Angular DI
      { provide: RESPONSE, useValue: res },
    ]
  }).then(html => res.send(html)).catch(next);
});

// auth.service.ts – reads cookie on server, localStorage in browser
getToken(): string | null {
  if (isPlatformServer(this.#platformId)) {
    const cookies = this.#request?.headers?.cookie ?? '';
    return this.#parseCookie(cookies, 'auth_token');
  }
  return localStorage.getItem('auth_token');
}
```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Add SSR to an existing Angular standalone app using ng add @angular/ssr. Identify three places where localStorage/window/document is accessed directly. Add isPlatformBrowser guards. Verify the SSR server starts without errors.

Exercise 2 — Intermediate

Build a BlogListComponent that fetches posts from an API in ngOnInit. Implement TransferState in BlogService so the server-side fetch is not repeated in the browser. Verify /api/posts is called exactly once via the Network panel. Add a click handler saving to localStorage with isPlatformBrowser guard.

Exercise 3 — Advanced

Build a full SSR-compatible ThemeService that: reads preferred theme from cookie on server (using REQUEST token) so correct theme applies in server-rendered HTML (eliminating flash), reads from localStorage in browser on subsequent visits, exposes a toggleTheme() method guarded with isPlatformBrowser. Write a unit test mocking PLATFORM_ID as 'server' and verifying toggleTheme() does not throw.

Section 10 — Key Takeaways

- SSR runs your Angular app in Node.js, sends fully rendered HTML to the browser, then hydrates
- isPlatformBrowser / isPlatformServer are your guards — any browser-only API must be wrapped
- afterNextRender() is the modern hook for browser-only DOM initialisation — never runs on the server
- TransferState prevents duplicate HTTP requests — server embeds data in HTML, browser reads it instead of refetching
- withFetch() in provideHttpClient() is required — Node.js has no XMLHttpRequest
- Hydration mismatches (NG0500) happen when server and browser render different content

Problem	Solution
localStorage is not defined	isPlatformBrowser guard
Third-party lib crashes server	afterNextRender() + dynamic import()
Duplicate HTTP requests	TransferState or withHttpTransferCache()
HTTP fails in Node	provideHttpClient(withFetch())
Hydration mismatch	Remove Math.random()/Date.now() from render path, or ngSkipHydration
Auth guard infinite loop	Return true on server, guard only in browser

One-Line Memory Aid: SSR sends a photograph of your app instantly, then the live cast quietly takes the stage — your only job is making sure nothing in the photo requires a browser to draw it.

Section 11 — Thought-Provoking Question + Answer

Question: Angular SSR hydration attaches to the server-rendered DOM instead of destroying and re-rendering it — but what happens to user interactions that occur in the milliseconds between the server HTML painting and hydration completing?

Answer: This is the hydration gap problem. When the browser receives server-rendered HTML, it paints it immediately — but Angular's JavaScript bundle hasn't finished downloading. If a user clicks a button during this window, the click handler doesn't exist yet.

Angular addresses this through event replay (Angular 18, `withEventReplay()`). A tiny inline script in the server-rendered HTML captures all user events during the hydration gap and queues them. Once hydration completes and Angular attaches real event listeners, the queued events are replayed in order.

Enable with: `provideClientHydration(withEventReplay())` in `app.config.ts`.

Practical rule: Enable `withEventReplay()` in any SSR app where users might interact immediately. Design event handlers defensively — they may fire before `ngOnInit` has fully run on the client.

Section 12 — Related Topics to Learn Next

- **Node.js module system and `import.meta` (JS)** — SSR server uses ES modules natively
- **Declaration merging and ambient module types (TS)** — SSR tokens like `REQUEST/RESPONSE` often require custom type declarations
- **Angular Edge Rendering** — Angular 17+ supports non-Node runtimes (Cloudflare Workers, Deno Deploy)
- **Angular Incremental Hydration** — only parts of the page hydrate on demand, further improving TTI

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 27 — DI, Topic 36 — `HttpClient`, Topic 22 — Async Pipe, Topic 25 — Lifecycle Hooks, Topic 38 — Router, Topic 49 — CDK.

Topics this unlocks: Every prior topic becomes production-complete once you understand the SSR constraint. Angular Micro-Frontends. Web Performance Engineering (Core Web Vitals).

Production readiness assessment: A developer who understands SSR, platform guards, `TransferState`, and hydration is the capstone topic that separates developers who build Angular apps from those who build deployed, performant, SEO-optimised Angular applications.

Section 14 — Common Interview Questions

Question 1 (Mid-Senior): "What is hydration in Angular SSR and why does it matter?"

Strong answer: Hydration attaches Angular's event listeners to existing server-rendered DOM without destroying it; without hydration users see a flash of blank content; `provideClientHydration()` must be in `app.config.ts`; NG0500 mismatches caused by non-deterministic values; `ngSkipHydration` opts out specific components.

Weak answers miss: The distinction between SSR (fast first paint) and hydration (making it interactive without a flash).

Question 2 (Mid): "How would you prevent duplicate HTTP requests in Angular SSR?"

Strong answer: `TransferState` with `makeStateKey` embeds fetched data as JSON in server-rendered HTML; browser reads from `TransferState` on first access and removes key; Angular 17+ shortcut: `withHttpTransferCache()` automates this for all GET requests.

Weak answers miss: The `withHttpTransferCache()` shortcut.

Question 3 (Senior): "SSR app works fine locally with ng serve but crashes in production with localStorage is not defined. How do you diagnose and fix this?"

Strong answer: ng serve uses CSR dev server — SSR never runs; reproduce with ng build then node dist/server.mjs; search for `localStorage/sessionStorage/window./document.;` wrap with `isPlatformBrowser`; for third-party libs accessing window at import time: lazy-load inside `afterNextRender(() => import('library'))`.

Weak answers miss: Why ng serve hides SSR bugs and the third-party library lazy-loading pattern.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Testing SSR only with ng serve

Why it seems fine: ng serve starts instantly, all features work, all tests pass. What breaks: ng serve runs CSR mode — SSR server never invoked. Every `localStorage` access, unguarded window call, or third-party DOM library crashes the production server.

Fix: Add `npm run serve:ssr` to local development workflow. Add CI step that builds and starts SSR server and makes a test HTTP request.

Anti-Pattern 2 — Skipping `withFetch()` and wondering why HTTP fails

```
// WRONG
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(), // Missing withFetch()
  ]
};
// Node.js has no XMLHttpRequest – every HTTP call throws
// ReferenceError: XMLHttpRequest is not defined
```

Fix: `provideHttpClient(withFetch())` — always, works in both browser and Node 18+.

Anti-Pattern 3 — Using `ngSkipHydration` as a blanket fix for hydration mismatch warnings

Why it seems fine: Hydration warnings disappear immediately. App works. What breaks: `ngSkipHydration` forces Angular to destroy server-rendered DOM and re-render on client — users see flash of missing content. LCP and CLS scores regress to CSR levels despite SSR overhead.

Fix: Identify and remove `Math.random()`, `Date.now()`, or browser-conditional rendering from initial render path. Reserve `ngSkipHydration` for genuinely non-deterministic components.

Section 16 — Mental Model Summary

Core concept: Angular SSR renders your app to HTML in Node.js, sends it to the browser for an instant first paint, then hydrates — attaching Angular to the existing DOM — so users see content immediately while the app becomes interactive in the background.

Key rules:

1. Guard every localStorage/window/document access with `isPlatformBrowser(inject(PLATFORM_ID))`
2. Use `afterNextRender()` for browser-only DOM initialisation — never runs on the server
3. Add `withFetch()` to `provideHttpClient()` — Node.js has no XMLHttpRequest
4. Use `TransferState` or `withHttpTransferCache()` to prevent duplicate HTTP requests
5. Always test SSR with `ng build && node dist/.../server.mjs` — `ng serve` never exposes SSR bugs
6. Use `ngSkipHydration` only as a last resort — causes visible content flash and defeats SSR for that subtree

THE SINGLE MOST IMPORTANT RULE: `ng serve` hides every SSR bug — always test with the actual server build before deploying; every localStorage or window access that isn't guarded will crash your production server.

TOPIC 51 — NgRx Core Concepts (Store, Actions, Reducers, Selectors)

NgRx State Management | Intermediate | Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The individual pieces (actions, reducers, selectors, store) are each simple; the difficulty is understanding why they are kept deliberately separate and how unidirectional data flow makes the whole system more predictable than the sum of its parts.

Angular Relevance: Critical — NgRx is the dominant state management solution in professional Angular teams; understanding its core mental model is a prerequisite for every topic from Topic 52 onward, and misunderstanding it is the single most common source of unmaintainable Angular codebases at scale.

Section 1 — Explanation

The Bank Ledger Analogy

Imagine a bank. The bank's vault holds the current balance (the Store — one authoritative state object). You cannot walk into the vault and change the number yourself. Instead, you fill out a deposit slip or withdrawal form (an Action — a plain description of what you want to happen, with no logic in it). A teller takes your form, reads the current balance, applies the transaction according to strict rules, and writes a new balance in the ledger (the Reducer — a pure function that takes current state + action and returns new state). At any time, you can ask the bank for a statement (a Selector — a memoised query that reads a specific slice of the ledger without exposing the raw vault).

The Four Core Pieces

Store — A single BehaviorSubject-backed object holding your entire application state tree. Components never mutate it directly.

Actions — Plain TypeScript objects with a mandatory type string property. Created with `createAction()`. They are descriptions of events, not commands.

Reducers — Pure functions: `(currentState, action) => newState`. Must be synchronous, have no side effects, and always return a new state object.

Selectors — Pure functions that derive data from the store. Created with `createSelector()`. Memoised — if the input slice hasn't changed, the projector doesn't re-run.

Unidirectional Data Flow

Component dispatches Action → Reducer computes new State → Store emits new State → Selector derives slice → Component receives updated data → (cycle repeats)

Piece	Created with	Role	Pure?
Store	<code>provideStore(reducers)</code>	Holds all state	N/A

Piece	Created with	Role	Pure?
Action	<code>createAction(type, props)</code>	Describes an event	Yes — no logic
Reducer	<code>createReducer(initial, on(...))</code>	Computes new state	Yes — must be
Selector	<code>createSelector(inputs, projector)</code>	Reads/derives state	Yes — memoised

Section 2 — Connection to Prior Concepts

- **Topic 44 — State Management Patterns:** NgRx is the industrial-scale implementation of the store pattern — private state, public read-only projections, write-only methods. NgRx formalises each into its own named piece.
- **Topic 20 — RxJS Subscription Management & Topic 46 — RxJS Operators:** The Store is a stream — `store.select(selector)` returns an Observable. Everything from Topics 20 and 46 applies directly.
- **Topic 22 — Async Pipe:** Selectors return Observables — the async pipe is the standard way to consume them in templates.
- **Topic 24 — OnPush Change Detection:** NgRx selectors emit only when their memoised output changes — pairs perfectly with OnPush.
- **Topic 14 — readonly & Immutability:** Reducers must return new state objects, never mutate existing ones — immutability is a correctness requirement in NgRx.

Most directly extends: Topic 44 — State Management Patterns.

Section 3 — Code Example

Plain TS — Core reducer concept

```
// The reducer pattern before NgRx
interface CounterState { count: number; }
const initialState: CounterState = { count: 0 };
function counterReducer(state = initialState, action: { type: string; amount?: number }) {
  switch (action.type) {
    case '[Counter] Increment':
      return { ...state, count: state.count + 1 }; // NEW object – never mutate
    case '[Counter] Add':
      return { ...state, count: state.count + (action.amount ?? 0) };
    default:
      return state; // ALWAYS return current state for unknown actions
  }
}
const state1 = counterReducer(undefined, { type: '@@INIT' }); // { count: 0 }
const state2 = counterReducer(state1, { type: '[Counter] Increment' }); // { count: 1 }
```

Actions — [Source] Event naming convention

```
// users/store/users.actions.ts
import { createAction, props } from '@ngrx/store';

// Page actions – dispatched by components
export const loadUsersPageOpened = createAction('[Users Page] Page Opened');
export const userSearched = createAction('[Users Page] User Searched', props<{
  query: string }>());

// API actions – dispatched by Effects after HTTP calls
export const loadUsersSuccess = createAction('[Users API] Load Users Success',
  props<{ users: User[] }>());
export const loadUsersFailure = createAction('[Users API] Load Users Failure',
  props<{ error: string }>());
```

Reducer

```
// users/store/users.reducer.ts
import { createReducer, on } from '@ngrx/store';

export interface UsersState {
  users: User[];
  loading: boolean;
  error: string | null;
  searchQuery: string;
}

const initialState: UsersState = { users: [], loading: false, error: null,
  searchQuery: '' };

export const usersReducer = createReducer(
  initialState,
  on(UsersActions.loadUsersPageOpened, state => ({ ...state, loading: true, error:
    null })),
  on(UsersActions.userSearched, (state, { query }) => ({ ...state, searchQuery: query
    })),
  on(UsersActions.loadUsersSuccess, (state, { users }) => ({ ...state, users,
    loading: false, error: null })),
  on(UsersActions.loadUsersFailure, (state, { error }) => ({ ...state, loading:
    false, error })))
);
```

Selectors

```
// users/store/users.selectors.ts
import { createFeatureSelector, createSelector } from '@ngrx/store';

const selectUsersState = createFeatureSelector<UsersState>('users');
export const selectAllUsers = createSelector(selectUsersState, state =>
  state.users);
export const selectUsersLoading = createSelector(selectUsersState, state =>
  state.loading);
export const selectSearchQuery = createSelector(selectUsersState, state =>
  state.searchQuery);

// Composed selector – projector only reruns when users OR searchQuery changes
export const selectFilteredUsers = createSelector(
  selectAllUsers,
```

```

selectSearchQuery,
(users, query) => {
  if (!query.trim()) return users;
  const lower = query.toLowerCase();
  return users.filter(u => u.name.toLowerCase().includes(lower));
}
);

```

Registration and Component

```

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideStore({}),
    provideStoreDevtools({ maxAge: 25, logOnly: false }),
  ]
};

// Feature route registration
export const usersRoutes: Routes = [{
  path: '',
  providers: [provideState('users', usersReducer)],
  component: UsersPageComponent,
}];

// Component
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, ... })
export class UsersPageComponent implements OnInit {
  readonly #store = inject(Store);
  filteredUsers$ = this.#store.select(selectFilteredUsers);
  loading$ = this.#store.select(selectUsersLoading);
  ngOnInit(): void {
    this.#store.dispatch(UsersActions.loadUsersPageOpened());
  }
  onSearch(query: string): void {
    this.#store.dispatch(UsersActions.userSearched({ query }));
  }
}

```

Section 4 — Before & After Refactor

WRONG — Service-based state with direct mutation

```

@Injectable({ providedIn: 'root' })
export class UsersService {
  users: User[] = []; // public mutable – anything can change this
  loadUsers(): void {
    this.http.get<User[]>('/api/users').subscribe(users => {
      this.users = users; // direct mutation – no other component knows
    });
  }
}

```

```
// Component A: this.userService.users.push(newUser); // silent, no sync
// Component B: reads stale data, OnPush never updates
```

CORRECT — NgRx unidirectional flow

```
// Dispatch an action – no logic, just a description
this.store.dispatch(userAdded({ user: newUser }));
// Reducer handles it – pure, testable, single place
on(userAdded, (state, { user }) => ({
  ...state,
  users: [...state.users, user] // new reference – OnPush updates all subscribers
}))
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Dispatching actions that describe commands instead of events

Wrong: `export const fetchUsers = createAction('[Users] Fetch Users')`

Right: `export const loadUsersPageOpened = createAction('[Users Page] Page Opened')`

Angular consequence: Command-style actions create tight coupling. Event-style actions allow multiple reducers and effects to react independently.

Mistake 2 — Mutating state inside a reducer

Wrong: `state.users = users; return state;` — returning same reference.

Right: `return { ...state, users, loading: false }`

Angular consequence: Selectors won't emit new values. OnPush components freeze. DevTools shows action dispatched but state diff is empty.

Mistake 3 — Selecting from store without using the async pipe

Wrong: `this.store.select(selectAllUsers).subscribe(users => { this.users = users; })` — no cleanup.

Right: `users$ = this.store.select(selectAllUsers)` — use async pipe in template.

Angular consequence: Zombie subscriptions accumulate — every created and destroyed component leaves an active subscription.

Section 6 — Do's and Don'ts

DO	DON'T
Name actions as past-tense events with [Source] prefix	Name actions as commands (fetchData, setLoading)
Always spread state in reducers: <code>{ ...state, changed: value }</code>	Mutate state properties directly inside reducers
Enable runtime immutability checks in development	Skip runtime checks and discover mutations as mysterious bugs

DO	DON'T
Use <code>createFeatureSelector</code> + <code>createSelector</code> for all state access	Access <code>store.select(state => state['featureName'].users)</code> inline
Register feature state lazily with <code>provideState()</code> in route providers	Register all feature reducers upfront in <code>provideStore()</code>
Use <code>ChangeDetectionStrategy.OnPush</code> on every component using the store	Use default change detection — it wastes render cycles

Section 7 — Debugging Tips

Bug 1 — Component doesn't update after dispatching an action

Checklist: (1) DevTools — confirm action dispatched and state diff shows change. (2) If no diff: mutation in reducer — enable `strictStateImmutability`. (3) If diff but no update: manual subscription without `markForCheck`. (4) Selector returning same reference from projector despite different data.

Bug 2 — `createFeatureSelector` returns undefined

Cause: Feature key string mismatch. `provideState('users', ...)` key must exactly match `createFeatureSelector('users')`. Check with: `store.pipe(take(1)).subscribe(state => console.log(Object.keys(state)))`.

Bug 3 — Action dispatched but reducer never handles it

Cause: The `on()` handler references a different action creator. In DevTools, check exact type string. Search codebase for that string — must be exactly one `createAction` with that type. Two `createAction` calls with same type string cause silent cross-file collisions.

Section 8 — Real-World Applications

Scenario 1 — Shopping Cart with Persistent State

```
// Reducer handles add/remove with immutable updates
on(itemAddedToCart, (state, { productId, quantity }) => {
  const existing = state.items.find(i => i.productId === productId);
  if (existing) {
    return { ...state, items: state.items.map(i =>
      i.productId === productId ? { ...i, quantity: i.quantity + quantity } : i
    )};
  }
  return { ...state, items: [...state.items, { productId, quantity }] };
}),
on(itemRemovedFromCart, (state, { productId }) => ({
  ...state, items: state.items.filter(i => i.productId !== productId)
})),
on(cartCleared, () => initialState) // reset to initial
```

Scenario 2 — Dashboard with Multiple Independent Data Slices

Each feature (profile, notifications, activity) has its own state slice registered lazily. Dashboard dispatches one action (dashboardPageOpened). Three independent Effects handle the three API calls. Three selectors provide data to three child components. All with OnPush — each component re-renders only when its specific selector emits.

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a standalone NgRx counter feature with actions for increment, decrement, and reset. Define a reducer handling all three. Create selectors for the current count and whether it is negative. Build a CounterComponent using async pipe for all template bindings.

Exercise 2 — Intermediate

Build a BookmarkService feature with NgRx. State holds an array of bookmarked article IDs, a maxBookmarks limit (default 10), and a limitReached boolean. Write reducers for bookmarkAdded and bookmarkRemoved. Create a composed selectBookmarkStatus selector. Demonstrate the projector only reruns when inputs actually change.

Exercise 3 — Advanced

Build a SearchState feature managing: search query, results array, pagination (current page, total pages, page size), and loading flag. Use [Source] Event naming. Write reducers for pagination navigation without losing current results. Write selectSearchViewModel returning all data a component needs in one object, and selectCurrentPageResults slicing results to current page.

Section 10 — Key Takeaways

- The Store is a single immutable state tree — components read through selectors, write by dispatching actions
- Actions are plain event descriptions — past tense, [Source] Event Name, no logic
- Reducers are pure functions — (state, action) => newState, always spread, never mutate
- Selectors are memoised queries — recompute only when input slice changes by reference
- Unidirectional flow makes state changes predictable, auditable, and time-travel debuggable

Concept	Rule	Common violation
Actions	Describe events, not commands	setLoading, fetchData style names
Reducers	Always return new reference	state.x = y; return state
Selectors	Compose from smaller selectors	Inline state => state.feature.deep.value
Store registration	Lazy with provideState() in routes	Eager in provideStore({featureReducer})

One-Line Memory Aid: Components describe what happened (Actions), reducers decide what changes (pure State), selectors read what exists (memoised Queries) — nobody touches the vault directly.

Section 11 — Thought-Provoking Question + Answer

Question: NgRx forces you to define actions, reducers, and selectors in separate files for even the simplest state change. Is this overhead ever justified, and how do you decide when NgRx is the right tool versus a simpler Signal-based service?

Answer: The overhead is real and the question is important. NgRx's verbosity is a deliberate trade of writing effort for reading clarity at scale.

NgRx is justified when: Multiple components share the same state; state changes have side effects that need to be auditable; the team is larger than three people; state transitions are complex (multi-step wizards, optimistic updates, real-time feeds).

Signal-based service is the right choice when: State is local to a single component tree; state is simple enough one developer can hold it in their head; the team is moving fast and verbosity genuinely slows delivery.

Practical rule: Default to Signal services for feature-local state. Reach for NgRx global Store when state is shared across multiple routes, when side effects need structure, or when the team needs a common contract for state changes.

Section 12 — Related Topics to Learn Next

- **Immutable update patterns (JS)** — spread and structural sharing explain why NgRx reducers work the way they do
- **Discriminated unions (TS)** — NgRx actions are discriminated unions; this explains how `on()` achieves type inference
- **Topic 52 — NgRx Effects:** Handle all async operations that actions trigger
- **Topic 53 — NgRx Selectors In Depth:** Memoisation, composition, `createSelectorFactory`
- **Topic 55 — NgRx Component Store:** When global store is too heavy

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 14 — Immutability, Topic 20 — RxJS Subscriptions, Topic 22 — Async Pipe, Topic 24 — OnPush CD, Topic 44 — State Management Patterns.

Topics this unlocks: Topics 52–55, 57, 58, 59 — all NgRx topics depend on this foundation.

Production readiness assessment: A developer who understands the four core concepts, names actions correctly, never mutates state, and uses selectors with async pipe is ready to contribute to any NgRx codebase.

Section 14 — Common Interview Questions

Question 1 (Junior-Mid): "Explain the role of each piece in NgRx and why they are kept separate."

Strong answer: Store = single source of truth, read-only externally; Actions = plain event descriptions enabling any part of app to react; Reducers = pure functions, only place state changes, enabling time-travel; Selectors = memoised derived data, decouple components from state shape. Separation enables independent testability and allows multiple reducers/effects to react to one action.

Weak answers miss: Why separation matters — not just what each piece does.

Question 2 (Mid): "What does 'pure function' mean in the context of NgRx reducers, and what breaks if a reducer isn't pure?"

Strong answer: Pure = same inputs always produce same output, no side effects. Breaks if impure: time-travel debugging replays actions — `Date.now()/Math.random()` causes divergence; mutation breaks selector memoisation; async operations break synchronous guarantee. `strictStateImmutability` catches violations.

Weak answers miss: The time-travel debugging consequence.

Question 3 (Senior): "Should you dispatch one action per user interaction or separate actions per state change?"

Strong answer: Neither is entirely right. One action per event — one action can be handled by multiple `on()` handlers across multiple feature reducers simultaneously. "One action per state change" leads to action dispatch chains (anti-pattern). The NgRx team's guidance: actions are events, not commands.

Weak answers miss: The multi-reducer aspect — one action can trigger changes across multiple independent feature reducers.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — "God actions" that carry the full new state

```
// WRONG
export const stateUpdated = createAction('[App] State Updated', props<{ newState:
Partial<UsersState> }>());
on(stateUpdated, (state, { newState }) => ({ ...state, ...newState }))
// DevTools shows '[App] State Updated' for everything – meaningless audit trail
```

Fix: One specific action per specific event. The verbosity is the feature, not the bug.

Anti-Pattern 2 — Selecting deeply nested state inline in components

```
// WRONG – inline lambda, no memoisation
userName$ = this.store.select(
  state => state['users']['entities'][this.userId]?.['profile']?.['displayName']
);
// New function reference every CD cycle – NgRx cannot memoise it
// Re-evaluates on EVERY store emission regardless of relevance
```

Fix: Always use `createFeatureSelector` + `createSelector` in a dedicated selectors file.

Anti-Pattern 3 — Dispatching actions in sequence from a component to orchestrate a workflow

```
// WRONG
onSubmitOrder(): void {
  this.store.dispatch(validateCart());
  this.store.dispatch(reserveInventory());
  this.store.dispatch(processPayment()); // fires even if validateCart failed
  this.store.dispatch(sendConfirmationEmail());
}
```

Fix: Dispatch one action ([Order Page] Order Submitted). Effects handle the conditional workflow — each step dispatches the next only on success.

Section 16 — Mental Model Summary

Core concept: NgRx enforces a strict one-way cycle where components dispatch event descriptions (Actions), pure functions compute new state (Reducers), a single object holds all state (Store), and memoised queries derive component data (Selectors) — nobody touches the state directly.

Key rules:

1. Actions describe past events: [Source] Event Happened — never commands
2. Reducers must be pure: always { ...state, changed } — never state.x = y
3. Selectors must be composed with createSelector — never inline lambdas in components
4. Register feature state lazily in route providers with provideState()
5. Always pair with ChangeDetectionStrategy.OnPush
6. Enable strictStateImmutability in development

THE SINGLE MOST IMPORTANT RULE: Never mutate state in a reducer — always return a new object with spread syntax; mutation silently breaks selector memoisation, freezes OnPush components, and corrupts DevTools time-travel, all with no error message.

TOPIC 52 — NgRx Effects

NgRx Async Middleware | Intermediate-Advanced | Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate-Advanced — The concept is straightforward (listen to actions, do async work, dispatch new actions), but the operator choice has silent, destructive consequences in production that are non-obvious until you know exactly what each one cancels.

Angular Relevance: Critical — Every real NgRx app has HTTP calls, navigation, and error handling that must live outside reducers; Effects are the only correct place for all of it, and getting the operator choice wrong is the most common source of data loss bugs in production NgRx applications.

Section 1 — Explanation

The Personal Assistant Analogy

Imagine your NgRx Store is a CEO. The CEO only makes decisions based on information (reducers computing state). The CEO never picks up the phone or fetches documents — that's beneath the role.

So the CEO has a personal assistant (an Effect). The assistant listens in on all meetings (listens to dispatched actions), and whenever a relevant topic comes up — "we need the Q3 report" — the assistant quietly leaves the room, makes the phone calls, fetches the document, and returns with either "here's the report" (`loadReportsSuccess`) or "the server was down" (`loadReportsFailure`). The CEO (reducer) then makes a decision based on that new information.

The assistant never changes the CEO's calendar directly. They always bring back a formal memo (dispatch a new action). This is the Effects contract.

Operator	Behaviour	Use case	Never use for
<code>switchMap</code>	Cancels previous, uses latest	Search, autocomplete, navigation	POST/PUT/DELETE
<code>concatMap</code>	Queues, strict order	Sequential writes, ordered operations	High-frequency events
<code>exhaustMap</code>	Ignores new while busy	Login, form submit (prevent double-submit)	Operations that must all complete
<code>mergeMap</code>	All concurrent, no cancel	Independent parallel reads	Ordered operations

Section 2 — Connection to Prior Concepts

- **Topic 51 — NgRx Core Concepts:** Effects complete the NgRx cycle — actions flow in, Effects react, new actions flow back out.

- **Topic 46 — RxJS Operators in Depth:** The four flattening operators from Topic 46 apply here with exactly the same cancellation behaviours — the stakes are higher because wrong choice silently drops writes.
- **Topic 36 — HttpClient & API Integration:** Effects are where HttpClient calls live in an NgRx app — the service layer from Topic 36 is called from inside Effects.
- **Topic 18 — Error Handling:** Every Effect that makes an HTTP call must catch errors — an unhandled error kills the Effect Observable permanently.

Most directly extends: Topic 46 — RxJS Operators in Depth (same operators, higher-stakes context).

Section 3 — Code Example

Full Effects Class

```
// users/store/users.effects.ts
import { Injectable, inject } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { of } from 'rxjs';
import { switchMap, map, catchError, exhaustMap, tap, concatMap } from 'rxjs/operators';

@Injectable()
export class UsersEffects {
  readonly #actions$ = inject(Actions);
  readonly #usersApi = inject(UsersApiService);
  readonly #router = inject(Router);

  // switchMap: latest request wins – OK for reads
  loadUsers$ = createEffect(() =>
    this.#actions$.pipe(
      ofType(UsersActions.loadUsersPageOpened),
      switchMap(() =>
        this.#usersApi.getUsers().pipe(
          map(users => UsersActions.loadUsersSuccess({ users })),
          catchError(error => // INSIDE inner Observable – critical!
            of(UsersActions.loadUsersFailure({ error: error.message })))
        )
      )
    )
  );

  // exhaustMap: ignore double-clicks while login is in flight
  login$ = createEffect(() =>
    this.#actions$.pipe(
      ofType(UsersActions.loginSubmitted),
      exhaustMap(({ credentials }) =>
        this.#usersApi.login(credentials).pipe(
          map(user => UsersActions.loginSuccess({ user })),
          catchError(error => of(UsersActions.loginFailure({ error: error.message })))
        )
      )
    )
  );
}
```

```

)
)
);
// concatMap: queue writes – never cancel a write
createUser$ = createEffect(() =>
this.#actions$.pipe(
ofType(UsersActions.createUserRequested),
concatMap(({ user }) =>
this.#usersApi.createUser(user).pipe(
map(created => UsersActions.createUserSuccess({ user: created })),
catchError(error => of(UsersActions.createUserFailure({ error: error.message })))
)
)
)
);
// { dispatch: false } – side effect only, no action dispatched back
navigateAfterLogin$ = createEffect(
() => this.#actions$.pipe(
ofType(UsersActions.loginSuccess),
tap(() => this.#router.navigate(['/dashboard']))
),
{ dispatch: false } // REQUIRED – omitting creates infinite dispatch loop
);
}

```

Registration

```

// users/users.routes.ts
export const usersRoutes: Routes = [{
path: '',
providers: [
provideState('users', usersReducer),
provideEffects([UsersEffects]), // register alongside feature state
],
component: UsersPageComponent,
}];

```

Section 4 — Before & After Refactor

WRONG — HTTP calls inside a component

```

@Component({})
export class UsersPageComponent implements OnInit {
readonly #http = inject(HttpClient);
ngOnInit(): void {
this.#http.get<User[]>('/api/users').subscribe(users => {
this.store.dispatch(UsersActions.loadUsersSuccess({ users }));
});
}
// No error handling – failed request leaves loading = true forever
// No cleanup – component destroyed, subscription still active

```

```
}  
}
```

CORRECT — Pure declarative component

```
@Component({})  
export class UsersPageComponent implements OnInit {  
  readonly #store = inject(Store);  
  loading$ = this.#store.select(selectUsersLoading);  
  users$ = this.#store.select(selectAllUsers);  
  error$ = this.#store.select(selectUsersError);  
  ngOnInit(): void {  
    this.#store.dispatch(UsersActions.loadUsersPageOpened());  
    // Effect handles: HTTP call, loading state, error handling, DevTools visibility  
  }  
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Placing catchError outside the inner Observable

Wrong: catchError at the outer stream level. One HTTP error kills the entire Effect permanently — it dies silently, no console warning, no recovery without page refresh.

Right: catchError inside the switchMap/concatMap inner Observable.

Angular consequence: The first HTTP error permanently kills the Effect. Every subsequent action is dispatched, hits the dead Effect, and silently does nothing.

Mistake 2 — Using switchMap for write operations

switchMap cancels the previous inner Observable when a new action arrives. For DELETE: user double-clicks, two deleteUserRequested actions dispatch, switchMap cancels the first DELETE mid-flight. Server state is undefined. Data integrity corrupted silently.

Fix: Use concatMap (preserve order) or exhaustMap (ignore duplicates) for all writes.

Mistake 3 — Forgetting { dispatch: false } on non-dispatching Effects

tap() returns the original action — NgRx tries to dispatch it again. loginSuccess dispatches loginSuccess which dispatches loginSuccess — infinite loop floods DevTools with hundreds of actions per second, crashes browser tab.

Fix: Any Effect ending with tap() needs { dispatch: false }.

Section 6 — Do's and Don'ts

DO	DON'T
Place catchError inside inner Observable (inside switchMap/concatMap)	Place catchError at outer stream — one error kills Effect forever
Use concatMap for write operations (POST/PUT/DELETE)	Use switchMap for writes — silently cancels in-flight mutations

DO	DON'T
Use exhaustMap for login and form submissions	Use mergeMap for login — allows unlimited concurrent requests
Add { dispatch: false } to every Effect ending with tap()	Omit { dispatch: false } — creates infinite dispatch loops
Register Effects lazily with provideEffects() in route providers	Register all Effects eagerly in app.config.ts
Call service methods from Effects	Call HttpClient directly in Effects — bypasses service error handling

Section 7 — Debugging Tips

Bug 1 — Effect stops working after the first error

Symptom: First HTTP call succeeds. After any error, Effect never fires again. DevTools shows actions dispatching but no HTTP calls in Network panel.

Cause: catchError is on the outer Observable. First error completes outer stream permanently.

Fix: Move catchError inside the switchMap/concatMap inner Observable.

Bug 2 — Infinite action dispatch loop

Symptom: DevTools shows same action repeating hundreds of times/second. Console: "Error: Maximum action dispatch limit reached."

Cause: Effect without { dispatch: false } uses tap() as terminal operator. Original action flows through tap() unchanged and NgRx dispatches it again.

Fix: Add { dispatch: false } as second argument to createEffect().

Bug 3 — Effect never fires at all

Causes: (1) Effects class never registered with provideEffects(). (2) ofType() references different action creator than dispatched — type string mismatch. (3) Effect registered in wrong route (not yet loaded).

Fix: Confirm provideEffects([YourEffects]) in route providers. Check exact type string in DevTools. Add temporary tap before ofType to confirm filter matching.

Section 8 — Real-World Applications

Scenario 1 — Optimistic Update with Rollback

```
// Reducer handles todoToggled immediately – no waiting for server
on(todoToggled, (state, { id, completed }) => ({
  ...state,
  todos: state.todos.map(t => t.id === id ? { ...t, completed } : t)
}))
// Effect confirms or rolls back
toggleTodo$ = createEffect(() =>
```



```

this.#actions$.pipe(
  ofType(todoToggled),
  concatMap(({ id, completed }) =>
    this.#todoApi.updateTodo(id, { completed }).pipe(
      map(() => todoToggleConfirmed({ id })),
      catchError(() =>
        // Roll back with OPPOSITE value
        of(todoToggleRolledBack({ id, completed: !completed })))
      )
    )
  )
);

```

Scenario 2 — Polling with Automatic Stop

```

pollMetrics$ = createEffect(() =>
  this.#actions$.pipe(
    ofType(MetricsActions.dashboardOpened),
    switchMap(() =>
      interval(5000).pipe(
        takeUntil(this.#actions$.pipe(ofType(MetricsActions.dashboardClosed))),
        switchMap(() =>
          this.#metricsApi.getLatest().pipe(
            map(metrics => MetricsActions.metricsLoaded({ metrics })),
            catchError(() => EMPTY) // silently ignore poll errors – continue polling
          )
        )
      )
    )
  );

```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create an Effect for a ProductsFeature that listens for [Products Page] Page Opened, calls ProductsApiService.getAll(), dispatches [Products API] Load Success or [Products API] Load Failure. Ensure catchError is correctly placed inside the inner Observable. Register in a lazy route.

Exercise 2 — Intermediate

Build a complete auth feature with three Effects: (1) login Effect using exhaustMap, (2) logout Effect using switchMap with tap clearing localStorage and dispatching logoutComplete, (3) navigate-after-login Effect using { dispatch: false } redirecting to /dashboard on loginSuccess.

Exercise 3 — Advanced

Build a file upload feature with an Effect that: accepts [Upload Page] Files Selected with an array of File objects, uses mergeMap to upload all files concurrently (each independent),

dispatches [Upload API] File Upload Progress with percentage updates, dispatches [UploadAPI] File Upload Complete per file, dispatches [Upload API] File Upload Failed per failed file —ensuring a single file failure does not cancel other uploads.

Section 10 — Key Takeaways

- Effects are the async middleware layer — listen to actions, perform side effects, dispatch new actions back
- The flattening operator choice is the most consequential decision in every Effect
- `catchError` must always be inside the inner Observable — outside = Effect dies on first error permanently
- `{ dispatch: false }` is required for any Effect that doesn't dispatch a new action
- Effects register alongside their feature state — lazy with `provideEffects()` in route providers

Scenario	Operator	Why
Load data on page open	<code>switchMap</code>	Latest request wins
Search / autocomplete	<code>switchMap</code>	Cancel stale searches
Create / update / delete	<code>concatMap</code>	Preserve order, never cancel writes
Login / form submit	<code>exhaustMap</code>	Ignore duplicates while in flight
Independent parallel uploads	<code>mergeMap</code>	All must complete, none cancel others

One-Line Memory Aid: Effects are the personal assistant who takes action descriptions, makes the phone calls, and brings back results — the operator they use decides whether they hang up on old calls or wait patiently in queue.

Section 11 — Thought-Provoking Question + Answer

Question: If `catchError` inside the inner Observable returns `of(failureAction(...))`, the Effect stream continues — but what happens to the component that dispatched the original action if the user retries the same operation that previously failed?

Answer: This is the stale error state problem. When an Effect catches an error and dispatches `loadUsersFailure({ error: 'Server error' })`, the reducer sets `error: 'Server error'` and `loading: false`. User clicks "Retry". The component dispatches `loadUsersPageOpened()` again.

The Effect fires correctly. But if the reducer for `loadUsersPageOpened` only sets `loading: true` and leaves `error: 'Server error'` in state — the component shows both a loading spinner AND the old error message simultaneously.

The fix is in the reducer:

```
on(UsersActions.loadUsersPageOpened, state => ({ ...state, loading: true, error: null })) //  
always clear error when retrying
```

Practical rule: For every action that initiates an async operation, ensure its `on()` handler sets loading: true AND clears error: null — treat it as a reset gate, not just a loading flag.

Section 12 — Related Topics to Learn Next

- **Topic 53 — NgRx Selectors In Depth:** Effects use `withLatestFrom(store.select(selector))` to read current state
- **Topic 54 — NgRx Entity:** Entity reducers handle the success actions Effects dispatch
- **Topic 58 — NgRx Testing:** Testing Effects requires marble testing with hot/cold observables

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 18 — Error Handling, Topic 36 — HttpClient, Topic 46 — RxJS Operators, Topic 51 — NgRx Core Concepts.

Topics this unlocks: Topic 53 — Selectors (`withLatestFrom`), Topic 54 — Entity, Topic 57 — Router Store, Topic 58 — Testing.

Production readiness assessment: A developer who places `catchError` correctly, chooses the right flattening operator for reads vs writes, and uses `{ dispatch: false }` correctly is ready to write production Effects.

Section 14 — Common Interview Questions

Question 1 (Mid): "Where should you place `catchError` in an NgRx Effect and why does placement matter?"

Strong answer: Must be inside the inner Observable (inside `switchMap/concatMap` callback).
Outer placement: one HTTP error completes outer Observable permanently, Effect dies silently.
Inner placement: error caught per-request, outer stream continues, Effect remains alive.
Returns `of(failureAction())` — `of()` creates completing Observable so inner stream ends cleanly.

Weak answers miss: Why the outer stream dying is silent — no console error, app just stops responding.

Question 2 (Mid-Senior): "When would you use `exhaustMap` vs `concatMap` vs `switchMap` in an NgRx Effect?"

Strong answer: `switchMap` = reads where latest-wins; `concatMap` = writes preserving order; `exhaustMap` = operations where duplicates must be ignored while one is in flight; `mergeMap` = independent parallel operations. Cardinal rule: never `switchMap` for writes — silently cancels in-flight mutations.

Weak answers miss: The specific production consequence of `switchMap` on a DELETE.

Question 3 (Senior): "Clicking 'Save' sometimes saves and sometimes doesn't, but only when clicking quickly. The Effect uses `switchMap`. What's happening?"

Strong answer: `switchMap` cancels previous inner Observable when new action arrives.
Double-clicking dispatches two save actions rapidly; first HTTP request cancelled by second.
Intermittent because timing-dependent. Fix: replace `switchMap` with `concatMap` (queue saves)

or exhaustMap (ignore second click while first in flight).

Weak answers miss: The intermittent nature explanation — why it sometimes works.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Effects that dispatch actions triggering other Effects in a chain

Three sequential Effects where one parallel forkJoin would do. Creates hidden coupling — changing Effect A's output action breaks Effect B silently. In DevTools, tracing a bug requires following 6+ action chain.

Fix: Use forkJoin or combineLatest in a single Effect for parallel data. Use concatMap with nested switchMap in one Effect when sequence is genuinely required.

Anti-Pattern 2 — Reading state with nested subscribe inside an Effect

```
// WRONG: nested subscription inside switchMap
switchMap(() => {
  let currentDoc;
  this.#store.select(selectCurrentDocument).pipe(take(1))
    .subscribe(doc => currentDoc = doc); // nested subscribe – memory leak risk
  return this.#api.save(currentDoc!).pipe(...);
})
// CORRECT: withLatestFrom
withLatestFrom(this.#store.select(selectCurrentDocument)),
switchMap([action, currentDoc] => this.#api.save(currentDoc).pipe(...))
```

Anti-Pattern 3 — Single catch-all Effect for multiple action types with complex branching

Why it seems fine: Fewer Effect properties, feels DRY. What breaks: switchMap now cancels ANY load when ANY other load fires — loading users cancels in-flight product loads. Impossible to unit test cleanly.

Fix: One Effect per action type per concern. Each independently testable, replaceable, debuggable.

Section 16 — Mental Model Summary

Core concept: Effects are NgRx's dedicated async layer — they listen to the action stream, perform side effects (HTTP, navigation, analytics) outside the pure reducer, and dispatch new actions back into the store to complete the cycle.

Key rules:

1. catchError always inside inner Observable — outside = Effect dies on first error, permanently, silently
2. switchMap for reads (latest wins), concatMap for ordered writes, exhaustMap for deduplicated submissions, mergeMap for independent parallel operations
3. Any Effect ending with tap() must have { dispatch: false } — omitting creates infinite dispatch loop

4. Register Effects lazily with `provideEffects([EffectsClass])` in route providers
5. Read current state in Effects with `withLatestFrom(store.select(...))` — never nested subscribe inside Effect

THE SINGLE MOST IMPORTANT RULE: `catchError` must be inside the `switchMap/concatMap` inner Observable — one misplaced `catchError` on the outer stream silently kills the entire Effect after the first error, with no console warning and no way to recover without a page refresh.

TOPIC 53 — NgRx Selectors In Depth

NgRx Selectors | Intermediate-Advanced | Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate-Advanced — The basic createSelector API is simple; the difficulty is understanding why memoisation works the way it does, what breaks it, how to compose selectors correctly at scale, and how poorly written selectors silently destroy the performance advantages NgRx is supposed to provide.

Angular Relevance: Critical — Selectors are the most performance-critical piece of NgRx; a single incorrectly written selector can cause every OnPush component in your app to re-render on every action dispatch, completely negating the change detection optimisations you paired NgRx with in the first place.

Section 1 — Explanation

The Spreadsheet Formula Analogy

Imagine a large Excel spreadsheet representing your application state. Raw data lives in cells (the store slices). You don't show users the raw cells — you show them computed formula cells that derive meaningful values: totals, filtered lists, formatted strings.

Excel is smart about recalculation. If cell A1 changes, only formulas that depend on A1 recalculate. A formula depending only on B1 and C1 doesn't rerun just because A1 changed. This is memoisation — recalculate only when the inputs you depend on actually change.

NgRx selectors are those formula cells. createSelector(inputA, inputB, projector) tells NgRx: "this selector depends on inputA and inputB — only rerun the projector when inputA or inputB produces a new value."

The Memoisation Chain

Store emits new state → Feature selector checks: is state['users'] a new reference? → If NO: return cached result, stop here → If YES: run next selector → selectAllUsers checks: is users array a new reference? → If NO: return cached → If YES: run projector → derive filtered list → cache result → emit

Every level can short-circuit. This is why composing selectors from smaller selectors is better than writing one large selector — more opportunities for early short-circuit.

API	Purpose	Key behaviour
createFeatureSelector<T>(key)	Get top-level feature slice	Entry point for all feature selectors
createSelector(inputs, projector)	Derive data from inputs	Memoised — projector only runs on new input references
createSelectorFactory(memoize)	Custom memoisation strategy	Used when referential equality is not appropriate

API	Purpose	Key behaviour
store.select(selector)	Subscribe to a selector	Returns Observable — emits only when result changes
selector.projector(args)	Call projector directly	Used in unit tests to test derivation logic

Section 2 — Connection to Prior Concepts

- **Topic 51 — NgRx Core Concepts:** Basic createSelector introduced there. This topic goes deeper — composition, memoisation mechanics, factory selectors.
- **Topic 24 — OnPush Change Detection:** Selectors and OnPush are designed as a pair. Memoised selectors emit only when derived value changes — making OnPush effectively free when selectors are written correctly.
- **Topic 23 — Angular Signals:** NgRx 17+ introduced selectSignal() — store.selectSignal(selector) returns a Signal, integrating NgRx state into the Signals reactivity system.
- **Topic 14 — readonly & Immutability:** Selectors depend on referential equality — which depends on reducers returning new references on change. Immutability is the prerequisite for memoisation.

Most directly extends: Topic 51 — NgRx Core Concepts.

Section 3 — Code Example

Plain TS — Memoisation mechanics in isolation

```
// What createSelector does under the hood
function createSelectorManual(input1, projector) {
  let lastInput1, lastResult;
  return (state) => {
    const newInput1 = input1(state);
    if (newInput1 === lastInput1 && lastResult !== undefined) {
      return lastResult; // Cache hit – same reference, skip projector
    }
    lastResult = projector(newInput1); // Cache miss – run projector
    lastInput1 = newInput1;
    return lastResult;
  };
}

const state = { users: [{ id: 1, name: 'Alice' }], loading: false };
const selectUsers = s => s.users;
const selectUserCount = createSelectorManual(selectUsers, users => users.length);
selectUserCount(state); // Cache miss – runs projector -> 1
selectUserCount(state); // Cache hit – returns 1 without running projector
// Unrelated state change – same users reference
const state2 = { ...state, loading: true };
```

```
selectUserCount(state2); // Cache hit – users unchanged, projector skipped
```

Full Selector Composition Hierarchy

```
// users/store/users.selectors.ts
// Level 1: Feature selector – entry point
const selectUsersState = createFeatureSelector<UsersState>('users');
// Level 2: Slice selectors – one per property, maximum short-circuit
export const selectAllUsers = createSelector(selectUsersState, s => s.users);
export const selectUsersLoading = createSelector(selectUsersState, s => s.loading);
export const selectSearchQuery = createSelector(selectUsersState, s =>
s.searchQuery);
export const selectSelectedUserId = createSelector(selectUsersState, s =>
s.selectedUserId);
// Level 3: Derived selectors – composed from Level 2
// Projector only reruns when users OR searchQuery changes – not when loading
changes
export const selectFilteredUsers = createSelector(
selectAllUsers,
selectSearchQuery,
(users, query) => {
if (!query.trim()) return users;
const lower = query.toLowerCase();
return users.filter(u => u.name.toLowerCase().includes(lower));
}
);
export const selectFilteredUserCount = createSelector(
selectFilteredUsers,
users => users.length // primitive return – memoised correctly by value
);
// Level 4: View model selector – one per component
export const selectUsersPageViewModel = createSelector(
selectFilteredUsers,
selectUsersLoading,
selectUsersError,
selectFilteredUserCount,
selectSearchQuery,
(users, loading, error, count, query) => ({
users, loading, error, count, query,
hasResults: users.length > 0,
isEmpty: !loading && users.length === 0 && !error
})
);
// Parameterised selector – factory pattern (create once per component instance)
export const selectUserById = (id: number) =>
createSelector(selectAllUsers, users => users.find(u => u.id === id) ?? null);
// Cross-feature selector
export const selectCurrentUserCartSummary = createSelector(
selectSelectedUser, // users feature
selectCartItems, // cart feature
```



```
(user, cartItems) => ({
  userName: user?.name ?? 'Guest',
  itemCount: cartItems.length,
})
);
```

Component consuming view model selector

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (vm$ | async; as vm) {
      <input [value]="vm.query" (input)="onSearch($event)" />
      <span>{{ vm.count }} results</span>
      @if (vm.loading) { <app-spinner /> }
      @for (user of vm.users; track user.id) {
        <app-user-card [user]="user" />
      }
    }
  `
})
export class UsersPageComponent {
  readonly #store = inject(Store);
  vm$ = this.#store.select(selectUsersPageViewModel); // ONE subscription
}
```

selectSignal() — NgRx 17+ Signal integration

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <!-- No async pipe needed – Signal integrates directly with template -->
    @if (vm().loading) { <app-spinner /> }
    @for (user of vm().users; track user.id) { <app-user-card [user]="user" /> }
  `
})
export class UsersPageComponent {
  readonly #store = inject(Store);
  vm = this.#store.selectSignal(selectUsersPageViewModel); // Signal, not Observable
}
```

Custom memoisation with createSelectorFactory

```
import { createSelectorFactory, resultMemoize } from '@ngrx/store';
import { isEqual } from 'lodash'; // deep equality
// Use when projector returns new array/object references with identical contents
const createDeepEqualSelector = createSelectorFactory(projector =>
  resultMemoize(projector, isEqual)
);
export const selectSortedUserIds = createDeepEqualSelector(
  selectAllUsers,
  // Sort creates new array reference every time – without deep equality
  // this would emit on every action touching users
);
```

```
users => [...users].sort((a, b) => a.id - b.id).map(u => u.id)
);
```

Section 4 — Before & After Refactor

WRONG — Inline selectors and template logic

```
// WRONG: Inline computation, multiple subscriptions
@Component({
  template: `
    <!-- Template method call – runs on EVERY CD cycle, never memoised -->
    <span>{{ getFilteredCount() }} users</span>
    <!-- Inline filtering in template – not memoised, not testable -->
    @for (user of (users$ | async)?.filter(u => u.name.includes(searchQuery)); track
    user.id) { }
  `
})
export class UsersComponent {
  users$ = this.store.select(state => state['users']['users']); // inline lambda!
  loading$ = this.store.select(state => state['users']['loading']); // inline lambda!
}
```

CORRECT — Composed view model selector

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, ... })
export class UsersComponent {
  readonly #store = inject(Store);
  vm$ = this.#store.select(selectUsersPageViewModel); // ONE subscription, all
  memoised
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Using inline lambda functions in store.select()

`users$ = this.store.select(state => state['users'].users.filter(u => u.active))` — new function reference every render. NgRx cannot memoise anonymous functions. Filter runs on EVERY store emission regardless of relevance.

Fix: Always use `createSelector` in a dedicated selectors file — stable function reference, fully memoised.

Mistake 2 — Creating new object or array references in a projector

A projector returning `{ count: users.length, hasAdmins: users.some(...) }` creates a new object literal on every call. New object = new reference = selector emits on every action touching users, even if count and hasAdmins are unchanged. `OnPush` stops working.

Fix: Return primitives where possible. For unavoidable objects: `createSelectorFactory` with `resultMemoize` and `deep equality`.

Mistake 3 — Recreating parameterised selectors inside component methods

`getUserSelector() { return createSelector(selectAllUsers, users => users.find(u => u.id === this.userId)); }` called in template. New selector instance on every CD cycle — empty memo cache each time, projector runs every render.

Fix: Create in `ngOnInit`: `this.user$ = this.store.select(selectUserById(this.userId));` — created once, stable reference.

Section 6 — Do's and Don'ts

DO	DON'T
Compose in layers: feature -> slice -> derived -> view model	Write one large selector doing all derivation in single projector
Return primitives from projectors when possible	Return new <code>{}</code> or <code>[]</code> literals — they break referential equality memoisation
Create parameterised selectors once in <code>ngOnInit</code>	Call <code>createSelector()</code> inside template-bound methods every cycle
Use <code>store.selectSignal(selector)</code> in Signal-based components	Use inline lambdas in <code>store.select(state => ...)</code> — cannot be memoised
Write one view model selector per component	Subscribe to five separate <code>store.select()</code> calls
Test projectors with <code>selector.projector(mockA, mockB)</code>	Set up full store just to test projector logic

Section 7 — Debugging Tips

Bug 1 — Component re-renders on every action even with `OnPush`

Cause: Selector returning new object/array reference on every call. Check with: `add tap(result => console.log('emitted:', result))` to selector Observable. If it logs on every action: projector returns new reference.

Fix: Return primitives, or split into primitive selectors, or use `createSelectorFactory` with deep equality.

Bug 2 — `createFeatureSelector` returns undefined causing selector chain to break

Cause: Feature key string mismatch between `provideState('users', reducer)` and `createFeatureSelector('users')`. Or lazy route not yet visited.

Fix: `store.pipe(take(1)).subscribe(state => console.log(Object.keys(state)))` to list all registered keys. Confirm exact match including case.

Bug 3 — Parameterised selector emitting stale data when ID changes

Cause: Selector created in `ngOnInit` with old ID, never recreated when Input changes. Angular reuses component instances in `@for` loops.

Fix: Use `ngOnChanges` to recreate selector when ID changes: `this.user$ = this.store.select(selectUserById(this.userId));`

Section 8 — Real-World Applications

Scenario 1 — Dashboard with Expensive Computed Metrics

```
// Expensive portfolio computation – only runs when positions OR prices change
// NOT when timeRange, loading, or any other unrelated state changes
export const selectPortfolioValue = createSelector(
  selectAllPositions,
  selectLivePrices,
  (positions, prices) =>
    positions.reduce((total, pos) => total + pos.shares * (prices[pos.ticker] ?? 0), 0)
);

export const selectPerformanceMetric = createSelector(
  selectPortfolioValue,
  selectTimeRange,
  (currentValue, timeRange) => {
    const baseline = getBaselineValue(timeRange);
    return {
      percentChange: ((currentValue - baseline) / baseline) * 100,
      trend: currentValue > baseline ? 'up' : 'down'
    };
  }
);
```

Scenario 2 — Multi-Step Form Validation State

```
export const selectStep1Valid = createSelector(selectFormState,
  s => s.personalInfo.name.length > 0 && s.personalInfo.email.includes('@'));
export const selectFormIsValid = createSelector(
  selectStep1Valid, selectStep2Valid, selectStep3Valid,
  (s1, s2, s3) => s1 && s2 && s3 // only reruns when any step validity changes
);

export const selectFormNavigationViewModel = createSelector(
  selectStep1Valid, selectStep2Valid, selectStep3Valid, selectFormIsValid,
  selectCurrentStep,
  (s1Valid, s2Valid, s3Valid, formValid, currentStep) => ({
    canProceed: currentStep === 1 ? s1Valid : currentStep === 2 ? s2Valid : s3Valid,
    canSubmit: formValid,
    stepsComplete: [s1Valid, s2Valid, s3Valid]
  })
);
```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a BookStore feature with state: books: Book[], selectedGenre: string, priceRange: { min, max }. Write four slice selectors, one composed selectFilteredBooks filtering by genre and price range, and one selectFilteredBookCount. Verify the filter projector only runs when books, selectedGenre, or priceRange changes — not on unrelated state changes.

Exercise 2 — Intermediate

Build a `selectProductPageViewModel` selector combining: selected product (by ID stored in state), related products (same category, excluding selected), whether product is in wishlist, and current cart item count. Write a unit test calling `selector.projector(mockProduct, mockRelated, false, 3)` directly without setting up a real store.

Exercise 3 — Advanced

`DataTableState` with: `rows`, `sortColumn`, `sortDirection`, `filterText`, `currentPage`, `pageSize`. Build complete selector hierarchy: level 1 slice selectors, level 2 `selectFilteredRows` and `selectSortedRows`, level 3 `selectPagedRows` and `selectTotalPages`. Build `selectTableViewModel` composing everything. Demonstrate that changing `currentPage` does not re-trigger filter or sort projectors.

Section 10 — Key Takeaways

- Selectors are memoised pure functions — cache last result, only recompute when input references change
- Memoisation works because reducers always return new references on change — same reference = nothing changed
- Compose in layers: feature → slice → derived → view model; more layers = more short-circuit opportunities
- Returning new object/array literals from projectors breaks memoisation — use primitives or `createSelectorFactory` with deep equality
- View model selectors combine all component needs into one subscription — cleaner components, fewer emissions

Rule	Correct	Wrong
Selector creation	<code>createSelector()</code> in selectors file	Inline lambda in <code>store.select(state => ...)</code>
Projector return type	Primitives or stable references	New <code>{}</code> or <code>[]</code> literals
Parameterised selector	Factory: <code>selectById(id)</code> in <code>ngOnInit</code>	Method: <code>getSelector()</code> in template
Component subscriptions	One view model selector	Multiple <code>store.select()</code> calls
Testing	<code>selector.projector(mockA, mockB)</code>	Full store setup for projector logic

One-Line Memory Aid: Selectors are Excel formula cells — they only recalculate when the cells they depend on change, and the more you compose them in layers, the more aggressively they can skip recalculation.

Section 11 — Thought-Provoking Question + Answer

Question: NgRx memoisation is based on referential equality — but what happens when you have a selector that sorts an array? Sorting always produces a new array reference even if the order is unchanged. Does this mean sorted selectors always break memoisation?

Answer: Yes — a naive sorted selector breaks memoisation completely because `[...users].sort()` always returns a new array reference.

Three production-grade solutions:

1. `createSelectorFactory` with `resultMemoize` and deep equality (lodash `isEqual`). Trade-off: deep equality is $O(n)$ — expensive for large arrays.
2. Sort only IDs (primitives) — cheap array of numbers to compare with deep equality.
3. Sort in the reducer on write — store pre-sorted data, selector is a simple slice reader with perfect memoisation. Best for static sort order.

Practical rule: For static sort order, sort in the reducer on write. For user-controlled sort, use `createSelectorFactory` with `resultMemoize` and deep equality, but be aware of the performance cost for large datasets.

Section 12 — Related Topics to Learn Next

- **Referential equality and structural sharing (JS)** — explains why `{...obj}` creates new reference even with identical values
- **`ReturnType<typeof selector>` (TS)** — extracts view model type from selector for component input types without duplication
- **Topic 54 — NgRx Entity:** Entity provides pre-built selectors (`selectAll`, `selectEntities`, `selectTotal`) — understanding their memoisation explains why Entity is more performant than manual array management
- **Topic 58 — NgRx Testing:** `selector.projector()` testing pattern — the practical application of selector knowledge

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 14 — Immutability, Topic 23 — Signals, Topic 24 — OnPush CD, Topic 51 — NgRx Core Concepts.

Topics this unlocks: Topic 54 — Entity (built-in selectors follow these patterns), Topic 55 — Component Store (same memoisation model), Topic 57 — Router Store, Topic 58 — Testing, Topic 59 — Best Practices.

Production readiness assessment: A developer who composes in layers, never uses inline lambdas, builds view model selectors per component, and understands why object literals in projectors break memoisation is writing production-quality NgRx that genuinely delivers the performance benefits the library promises.

Section 14 — Common Interview Questions

Question 1 (Mid-Senior): "How does NgRx selector memoisation work, and what can break it?"

Strong answer: Referential equality (===) comparison of input values between calls. If all inputs return same references, projector skipped and cached result returned. Works because reducers return new references on change. Breaks: inline lambdas in store.select() (new function reference); object/array literals in projectors (always new reference); mutations in reducers (same reference despite changed data). Fix: createSelectorFactory with resultMemoize and deep equality.

Weak answers miss: Connection to reducer immutability — memoisation depends entirely on reducers returning new references.

Question 2 (Mid): "What is a view model selector and why would you use one?"

Strong answer: Combines all state a specific component needs into one derived object. Benefits: one Observable, one emission per state change, all derivation in testable pure function. Trade-off: view model object is new reference on every emission — must ensure primitive inputs to control emission frequency. selectSignal(viewModelSelector) integrates with Signal templates.

Weak answers miss: The emission frequency trade-off.

Question 3 (Senior): "An OnPush component re-renders on every action despite using NgRx selectors. Walk through your diagnosis."

Strong answer: Step 1 — Angular DevTools profiler confirm actual re-renders. Step 2 — tap(v => console.log('emitted', v)) to confirm selector emits on every action. Step 3 — Check projector returns primitive or object literal. Step 4 — Check store.select() uses named selector not inline lambda. Step 5 — If projector returns object: use createSelectorFactory or split into primitive selectors. Step 6 — Check reducer for mutations.

Weak answers miss: The inline lambda case.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Deriving data in the component instead of in a selector

```
// WRONG: computation in subscription callback
this.store.select(selectAllUsers).subscribe(users => {
  this.users = users;
  this.activeUserCount = users.filter(u => u.active).length; // not memoised
  this.adminUsers = users.filter(u => u.role === 'admin'); // not memoised
});
// Every action touching users = two .filter() calls + two CD checks
```

Fix: Move both derivations into named selectors. Component holds one Observable, renders one vm\$ | async, contains zero computation.

Anti-Pattern 2 — Sharing a memoised selector instance across components with different parameters

Module-level parameterised selector used by multiple components — they share the same memo cache. Component A calls with ID 1 (caches). Component B calls with ID 2 — cache miss, caches ID 2. Component A renders again — cache miss (last was ID 2), runs projector again. Two components constantly invalidating each other.

Fix: Factory pattern — each component creates its own instance in `ngOnInit`: `this.user$ = this.store.select(selectUserById(this.userId));`

Anti-Pattern 3 — Bypassing selectors with direct store state access

```
// WRONG: direct store access inside Effect
this.store.pipe(take(1)).subscribe(state => {
  userName = state['users'].users.find(u => u.id === userId)?.name;
// String key bypasses TypeScript – renamed feature key = runtime undefined
// Nested subscribe = memory leak risk
});
// CORRECT: withLatestFrom with typed selector
withLatestFrom(this.#store.select(selectAllUsers)),
switchMap(([{ userId }, users]) => { ... })
```

Section 16 — Mental Model Summary

Core concept: NgRx selectors are memoised pure functions that short-circuit recomputation when their input references haven't changed — they are the performance engine of NgRx, and every decision about how you write them directly determines whether OnPush change detection actually works as intended.

Key rules:

1. Compose in layers: feature selector → slice selectors → derived selectors → view model selector
2. Projectors should return primitives where possible — object/array literals always produce new references
3. Never use inline lambdas in `store.select()` — use named selectors from a selectors file
4. Create parameterised selectors once per component instance (in `ngOnInit`) — not in template-bound methods
5. Use `store.selectSignal(selector)` in Signal-based components — eliminates async pipe
6. Test projectors directly with `selector.projector(mockA, mockB)` — pure functions need no store setup

THE SINGLE MOST IMPORTANT RULE: Returning a new object or array literal from a projector breaks memoisation — the selector emits on every action that touches its inputs, OnPush stops working, and your app renders as frequently as with default change detection — silently, with no warning.